BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HCM

ĐỒ ÁN CƠ SỞ

NGHIÊN CỨU NGUY CƠ MẤT AN TOÀN TRONG IOT

Giảng viên hướng dẫn: ThS. Nguyễn Trọng Minh Hồng Phước

Sinh viên thực hiện	Lớp	MSSV
1. Lê Gia Huy	22DTHH5	2280608204

TP. Hồ Chí Minh, 2025

LỜI MỞ ĐẦU

Trong bối cảnh cuộc cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ trên toàn cầu, công nghệ thông tin và truyền thông đã và đang đóng vai trò then chốt trong việc đổi mới phương thức sản xuất, quản lý và đời sống con người. Một trong những thành tựu nổi bật nhất trong giai đoạn này chính là Internet of Things (IoT) – một hệ sinh thái gồm các thiết bị, cảm biến, phần mềm và các hệ thống kết nối được với nhau thông qua Internet, cho phép thu thập, chia sẻ và xử lý dữ liệu theo thời gian thực.

IoT không chỉ mang đến nhiều tiện ích trong việc tự động hóa, tối ưu hóa năng suất lao động và giảm thiểu chi phí vận hành mà còn góp phần nâng cao chất lượng cuộc sống. Những ứng dụng của IoT có thể kể đến như: nhà thông minh (Smart Home), thành phố thông minh (Smart City), y tế thông minh (Smart Healthcare), nông nghiệp chính xác (Precision Agriculture), giao thông thông minh (Smart Transportation), và rất nhiều lĩnh vực khác. Sự phát triển này đang mở ra một kỷ nguyên mới của sự kết nối và tương tác giữa con người – thiết bị – môi trường.

Tuy nhiên, đi kèm với sự phát triển nhanh chóng và mức độ phổ biến rộng rãi của các thiết bị IoT là những nguy cơ tiềm ẩn về mất an toàn và an ninh mạng. Đặc điểm của các thiết bị IoT là thường có tài nguyên phần cứng hạn chế, ít được trang bị các biện pháp bảo mật mạnh mẽ, và thường xuyên bị bỏ qua trong quá trình cập nhật hoặc vá lỗi bảo mật. Bên cạnh đó, việc triển khai các hệ thống IoT không theo chuẩn mực bảo mật cụ thể, thiếu cơ chế giám sát hiệu quả và sự chủ quan của người dùng cũng góp phần làm gia tăng khả năng bị tấn công. Nhiều cuộc tấn công mạng lớn trong thực tế đã được thực hiện thông qua việc lợi dụng các thiết bị IoT bị cấu hình sai hoặc bảo mật kém, điển hình như cuộc tấn công botnet Mirai đã từng khiến hàng loạt hệ thống trên Internet tê liệt.

Nhận thức được tầm quan trọng của vấn đề, nhóm chúng em quyết định lựa chọn đề tài: “Nghiên cứu nguy cơ mất an toàn trong IoT” để thực hiện đồ án môn học Cơ sở ngành An ninh mạng. Đề tài hướng tới việc tìm hiểu tổng quan về hệ sinh thái IoT, phân tích các rủi ro bảo mật phổ biến trong các thiết bị và giao thức IoT, cũng như xây dựng một mô hình mô phỏng thực tế để kiểm thử các điểm yếu bảo mật. Thông qua việc sử dụng các công cụ như VirtualBox để tạo lập môi trường mô phỏng, Kali Linux để thực hiện các bài kiểm thử tấn công, Alpine Linux để đóng vai trò thiết bị IoT và máy chủ nhẹ, cùng với các nền tảng truyền thông phổ biến như MQTT Broker (Mosquitto) và Node-RED, nhóm sẽ triển khai và phân tích các tình huống tấn công điển hình như: rò rỉ thông tin, tấn công từ chối dịch vụ (DoS/DDoS), giả mạo thiết bị, chiếm quyền điều khiển hệ thống, v.v.

Mục tiêu chính của đồ án không chỉ là nhận diện các nguy cơ hiện hữu mà còn là bước đầu đề xuất các giải pháp bảo mật cơ bản để giảm thiểu rủi ro, từ đó nâng cao ý thức và năng lực ứng dụng kiến thức bảo mật trong thực tiễn phát triển và triển khai các hệ thống IoT. Đây cũng là một cơ hội quý báu để nhóm áp dụng các kiến thức lý thuyết đã học về hệ điều hành, mạng máy tính, giao thức truyền thông và an toàn thông tin, kết hợp với kỹ năng thực hành chuyên sâu để giải quyết một bài toán thực tế có tính thời sự và ứng dụng cao.

Với tinh thần học hỏi nghiêm túc, chủ động nghiên cứu và cộng tác nhóm chặt chẽ, nhóm chúng em cam kết sẽ cố gắng hoàn thành đề tài một cách đầy đủ, khoa học và chất lượng. Mặc dù trong quá trình thực hiện không tránh khỏi những khó khăn và hạn chế, nhưng với sự hướng dẫn tận tình của giảng viên cùng sự nỗ lực của cả nhóm, chúng em hy vọng đề tài sẽ đạt được những kết quả có ý nghĩa về mặt học thuật và thực tiễn.

LỜI CẢM ƠN

Trong bối cảnh công nghệ thông tin không ngừng phát triển và trở thành một phần không thể thiếu trong mọi lĩnh vực của đời sống hiện đại, vấn đề an ninh mạng ngày càng được quan tâm đúng mức. Đặc biệt, sự bùng nổ của Internet of Things (IoT) – một mạng lưới các thiết bị thông minh được kết nối với nhau thông qua Internet – đã mang đến nhiều tiện ích nhưng đồng thời cũng đặt ra những thách thức lớn về an toàn thông tin và bảo mật hệ thống.

Với mong muốn được tiếp cận và nghiên cứu sâu hơn về những rủi ro tiềm ẩn trong môi trường IoT, nhóm chúng em đã lựa chọn đề tài "Nghiên cứu nguy cơ mất an toàn trong IoT" để thực hiện đồ án Cơ sở ngành An ninh mạng. Trong quá trình thực hiện đề tài, chúng em đã có cơ hội vận dụng các kiến thức lý thuyết đã học vào mô hình thực tiễn, tìm hiểu các kỹ thuật tấn công và phòng thủ cơ bản, từ đó củng cố nền tảng chuyên môn và tư duy bảo mật cần thiết cho ngành học.

Em xin bày tỏ lòng biết ơn sâu sắc đến Thầy ThS. Nguyễn Trọng Minh Hồng Phước – người đã tận tình hướng dẫn, định hướng và góp ý xuyên suốt quá trình chúng em thực hiện đồ án. Sự hỗ trợ của thầy không chỉ giúp em vượt qua những khó khăn về kỹ thuật mà còn truyền cảm hứng để em nghiêm túc hơn trong việc học tập, nghiên cứu và phát triển kỹ năng nghề nghiệp trong lĩnh vực an ninh mạng.

Bên cạnh đó, em cũng xin gửi lời cảm ơn đến Khoa Công nghệ Thông tin và nhà trường đã tạo điều kiện học tập, cung cấp môi trường và tài nguyên cần thiết để nhóm có thể thực hiện tốt đề tài này. Đồng thời, em cũng xin ghi nhận sự đóng góp ý kiến từ các bạn học cùng lớp – những người đã cùng trao đổi, chia sẻ và hỗ trợ nhóm trong quá trình hoàn thiện sản phẩm.

Mặc dù đã nỗ lực hết sức, nhưng do kiến thức và kinh nghiệm thực tiễn còn hạn chế, bài báo cáo không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp từ thầy và các bạn để đề tài được hoàn thiện tốt hơn trong tương lai.

Chúng em xin chân thành cảm ơn!

Sinh viên thực hiện

Lê Gia Huy

NHIỆM VỤ ĐỒ ÁN

Trong khuôn khổ đồ án cơ sở ngành An ninh mạng với đề tài "Nghiên cứu nguy cơ mất an toàn trong IoT" , nhóm chúng em đã xác định rõ các nhiệm vụ cần thực hiện nhằm đạt được mục tiêu phân tích, đánh giá và đề xuất giải pháp bảo mật cho hệ thống IoT. Cụ thể, các nhiệm vụ chính của đồ án bao gồm:

1. **Xây dựng mô hình hệ thống IoT mô phỏng thực tế:**
   - **Thiết kế và triển khai hệ thống gồm ba thành phần chính:**
     + Client: Thiết bị IoT phát dữ liệu (ví dụ: cảm biến nhiệt độ giả lập).
     + Broker: Máy chủ trung gian sử dụng giao thức MQTT (Mosquitto).
     + Attacker: Máy tấn công (Kali Linux) mô phỏng các hành vi xâm nhập trái phép.
   - **Sử dụng VirtualBox để tạo môi trường ảo hóa với các máy ảo Ubuntu Server, Alpine Linux và Kali Linux phục vụ kiểm thử.**
2. **Thực hiện các kịch bản tấn công phổ biến trong môi trường IoT chưa được bảo mật:**
   - **Nghe lén dữ liệu (Sniffing):** Sử dụng Wireshark và tcpdump để thu thập và phân tích các gói tin MQTT không được mã hóa.
   - **Tấn công giả mạo (Man-in-the-Middle – MITM):** Chèn dữ liệu giả hoặc thay đổi nội dung khi chưa có mã hóa.
   - **Tấn công từ chối dịch vụ (DoS):** Gửi số lượng lớn kết nối hoặc dữ liệu đến broker để làm gián đoạn hoạt động.
   - **Tấn công brute-force mật khẩu:** Thử nghiệm các công cụ như Hydra để dò tìm mật khẩu MQTT broker khi không có giới hạn truy cập**.**
3. **Cấu hình và triển khai các biện pháp bảo mật cho giao thức MQTT:**
   - **Xác thực người dùng:** Thiết lập cơ chế xác thực qua tập tin mật khẩu (mosquitto_passwd).
   - **Mã hóa dữ liệu:** Tạo và áp dụng chứng chỉ số tự ký để cấu hình bảo mật TLS/SSL cho broker và client.
4. **Cập nhật mã nguồn thiết bị IoT (client):**

- Viết lại đoạn mã Python mô phỏng thiết bị IoT để hoạt động được cả trong môi trường không bảo mật (plaintext MQTT) và có bảo mật (MQTT qua TLS).

5. **Phân tích và so sánh giữa môi trường bảo mật và không bảo mật:**
   - Sử dụng công cụ giám sát mạng như Wireshark, tcpdump để quan sát sự thay đổi trong gói tin khi bật/tắt mã hóa.
   - Đánh giá hiệu năng hệ thống: Ghi nhận độ trễ, tài nguyên sử dụng và khả năng chống chịu tấn công.
6. **Đánh giá tổng thể và rút ra kết luận:**
   - Phân tích mức độ hiệu quả của các giải pháp bảo mật đã triển khai.
   - Xác định điểm mạnh – điểm yếu của từng mô hình và khả năng ứng dụng vào thực tiễn triển khai IoT trong doanh nghiệp hoặc hộ gia đình.

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

MỤC LỤC

LỜI MỞ ĐẦU .....

LỜI CẢM ƠN ..... **II**

NHIỆM VỤ ĐỒ ÁN ..... **III**

NHẬN XÉT CỦA GIẢNG VIÊN ..... **V**

MỤC LỤC ..... **VI**

DANH MỤC HÌNH ẢNH ..... **XI**

DANH MỤC BẢNG BIỂU ..... **XII**

CHƯƠNG 1: TỔNG QUAN **13**

1.1. Đặt vấn đề và động cơ nghiên cứu **13**

1.2. Khảo sát thực trạng **14**

1.2.1. Bức tranh tổng thể về an ninh trong IoT 14

1.2.2. Các cuộc tấn công thực tế từng xảy ra 15

1.2.3. Giao thức giao tiếp – điểm yếu phổ biến trong hệ thống IoT 15

1.2.4. Khảo sát hệ thống mô phỏng trong khuôn khổ đồ án 16

1.3. Đánh giá **16**

1.3.1. Ưu điểm của hệ thống IoT 16

1.3.2. Hạn chế và thách thức về bảo mật trong IoT 17

1.3.3. Định hướng nghiên cứu trong đồ án 18

1.4. Nhiệm vụ đồ án **18**

1.4.1. Nghiên cứu cơ sở lý thuyết về IoT và bảo mật 18

1.4.2. Thiết kế mô hình mô phỏng hệ thống IoT 19

1.4.3. Mô phỏng các cuộc tấn công điển hình vào hệ thống IoT 19

1.4.4. Triển khai và kiểm nghiệm các giải pháp bảo mật 19

1.5. Đối tượng và phạm vi ứng dụng **20**

1.5.1. Đối tượng ứng dụng 20

1.5.2. Phạm vi ứng dụng 20

1.6. Cấu trúc đồ án **21**

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT **23**

2.1. Tổng quan về Internet of Things (IoT) **23**

2.1.1. Khái niệm IoT 23

2.1.2. Các thành phần cơ bản trong hệ thống IoT 23
2.1.3. Kiến trúc hệ thống IoT 23
2.1.4. Liên hệ với hệ thống mô phỏng trong đồ án 24
2.2. Giao thức MQTT trong hệ thống IoT 26
2.2.1. Tổng quan về MQTT.....26
2.2.2. Kiến trúc và cơ chế hoạt động của MQTT 27
2.2.3. Ứng dụng thực tiễn của MQTT trong hệ thống IoT.....28
2.2.4. Các điểm yếu bảo mật của MQTT 28
2.2.5. Biện pháp tăng cường bảo mật cho MQTT.....29
2.2.6. Đánh giá tổng quan.....30
2.3. Các loại tấn công phổ biến trên IoT 30
2.3.1. Tổng quan về nguy cơ mất an toàn trong IoT 30
2.3.2. Tấn công nghe lén (Eavesdropping).....31
2.3.3. Tấn công giả mạo trung gian (Man-in-the-Middle – MITM) 32
2.3.4. Tấn công từ chối dịch vụ (DoS – Denial of Service) 32
2.3.5. Tấn công brute-force (Đoán mật khẩu) 33
2.3.6. Tấn công phát lại (Replay Attack).....33
2.3.7. Tấn công khai thác lỗ hổng phần mềm (Firmware Exploit).....33
2.3.8. Phân tích hành vi (Traffic Analysis) 33
2.3.9. Tổng hợp và phân loại mức độ nguy hiểm.....33
2.4. Giới thiệu các biện pháp bảo mật cơ bản 34
2.4.1. Xác thực người dùng (Authentication).....34
2.4.2. Mã hóa dữ liệu (Encryption) 35
2.4.3. Phân quyền truy cập (Access Control) 35
2.4.4. Giám sát và ghi log (Monitoring & Logging) 35
2.4.1. Các biện pháp bổ sung.....36
2.5. Tổng kết 36
2.6. Công cụ và phần mềm sử dụng trong mô phỏng 36
2.6.1. Oracel VirtualBox 36
2.6.2. Alpine Linux.....37
2.6.3. Kali Linux.....38
2.6.4. Eclipse Mosquitto (MQTT Broker).....38

2.6.5. Node-RED 39
2.6.6. Python + Paho MQTT 39
CHƯƠNG 3: CHƯƠNG 3: THIẾT KẾ HỆ THỐNG.....40
3.1. Kiến trúc tổng thể hệ thống mô phỏng 40
3.1.1. Mục tiêu thiết kế hệ thống.....40
3.1.2. Các thành phần chính trong hệ thống.....40
3.1.3. Mô hình mạng triển khai 41
3.1.4. Sơ đồ kiến trúc tổng thể hệ thống.....42
Dưới đây là sơ đồ minh họa kiến trúc hệ thống mô phỏng: 42
3.1.5. Tổng kết.....42
3.2. Thiết kế mô hình Client – Broker – Attacker 43
3.2.1. Thiết kế máy Client - Thiết bị IoT.....43
3.2.2. Thiết kế máy Broker (máy chủ MQTT) 44
3.2.3. Thiết kế máy Attacker (mô phỏng kẻ tấn công).....45
3.2.4. Mô hình tổng thể 46
3.3. Kết luận 46
3.4. Kịch bản tấn công và cách triển khai 46
3.4.1. Kịch bản 1: Tấn công nghe lén dữ liệu (Sniffing).....46
3.4.2. Kịch bản 2: Giả mạo thiết bị IoT (Impersonation Attack) 46
3.4.3. Kịch bản 3: Tấn công từ chối dịch vụ (DoS - Flooding).....47
3.4.4. Kịch bản 4: Tấn công brute-force topic (Topic Discovery) 47
3.4.5. Tóm tắt và đánh giá 48
3.5. Triển khai giải pháp bảo mật: Xác thực và mã hóa 48
3.5.1. Xác thực người dùng với Username/Password 48
3.5.2. Mã hóa kết nối bằng TLS (Transport Layer Security) 49
3.5.3. Kết hợp xác thực và mã hóa 50
3.5.4. So sánh MQTT mặc định vs MQTT bảo mật.....50
3.5.5. Kết luận.....51
3.6. Cấu hình thiết bị và cập nhật mã nguồn client 51
3.6.1. Cấu hình thiết bị client 51
3.6.2. Cập nhật mã nguồn client Python.....52
3.6.3. Kết quả mong đợi 54

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ 55
4.1. Vận hành trên localhost 55
4.1.1. Mô tả môi trường không bảo mật 55
4.1.2. Mô tả môi trường có bảo mật 55
4.1.3. So sánh tổng hợp 56
4.1.4. Kết luận sơ bộ 57
4.2. Đánh giá hiệu quả giải pháp bảo mật 57
4.2.1. Khả năng ngăn chặn tấn công 57
4.2.2. Độ an toàn và toàn vẹn dữ liệu 57
4.2.3. Bảo vệ tính toàn vẹn kết nối 58
4.2.4. Mức độ ảnh hưởng đến hiệu suất hệ thống 58
4.2.5. Tổng kết đánh giá 58
4.3. Phân tích độ trễ, phản hồi và tài nguyên sử dụng 62
4.3.1. Độ trễ truyền dữ liệu 62
4.3.2. Phản hồi hệ thống 62
4.3.3. Tài nguyên hệ thống sử dụng 63
4.3.4. Tổng kết hiệu năng 63
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 65
5.1. Kết luận 65
5.2. Hạn chế 66
5.2.1. Quy mô hệ thống mô phỏng còn nhỏ 66
5.2.2. Giải pháp bảo mật còn ở mức cơ bản 66
5.2.3. Thiếu các tiêu chí đánh giá định lượng chuyên sâu 66
5.2.4. Chưa triển khai hệ thống giám sát hoặc cảnh báo 67
5.2.5. Chưa tích hợp các lớp học máy (AI/ML) để phát hiện tấn công 67
5.3. Hướng phát triển trong tương lai 67
5.3.1. Mở rộng quy mô mô hình IoT 67
5.3.2. Tích hợp các giao thức IoT khác ngoài MQTT 67
5.3.3. Tăng cường các biện pháp bảo mật nâng cao 68
5.3.4. Triển khai hệ thống giám sát và cảnh báo sự cố 68
5.3.5. Ứng dụng trí tuệ nhân tạo (AI/ML) vào bảo mật IoT 68
5.3.6. Triển khai mô hình thử nghiệm thực tế 68

TÀI LIỆU THAM KHẢO.....69

DANH MỤC HÌNH ẢNH

Hình 2.1. Sơ đồ kiến trúc mô phỏng 26
Hình 2.2. Sơ đồ mô hình hoạt động 27
Hình 3.1. Cấu hình mạng nội bộ cho máy ảo 42
Hình 3.2. Sơ đồ minh họa hệ thống 42
Hình 3.3. Cấu hình script gửi dữ liệu nhiệt độ 44
Hình 3.4. Cấu hình Mosquitto 45
Hình 3.5. Cấu hình Mosquitto bảo mật 49
Hình 3.6. Lệnh tạo chứng chỉ và khóa 49
Hình 3.7. Cấu hình Mosquitto sử dụng TLS 50
Hình 3.8. Cập nhật cấu hình MQTT Client 50
Hình 3.9. Mã nguồn hoàn chỉnh của script nhiệt độ 53
Hình 4.1. Máy tấn công nghe lén được 59
Hình 4.2. Máy tấn công giả mạo thiết bị 59
Hình 4.3. Máy tấn công tấn công DoS 60
Hình 4.4. Bảo mật yêu cầu user – password 60
Hình 4.5. Bảo mật TLS – Yêu cầu chứng chỉ 61

DANH MỤC BẢNG BIỂU

Bảng 2.1. Các mức QoS 28
Bảng 2.2. Các thành phần trong MQTT 28
Bảng 2.3. Bảng giải pháp đề xuất 30
Bảng 2.4. Mức độ nguy hiểm của các loại tấn công trong IoT 34
Bảng 3.1. Sơ đồ kiến trúc mô hình Client – Broker – Attacker 46
Bảng 3.2. Bảng tóm tắt kịch bản tấn công 48
Bảng 3.3. Bảng so sánh giữa MQTT mặc định và bảo mật 51
Bảng 4.1. Bảng so sánh tổng hợp 56
Bảng 4.2. Tổng kết đánh giá .....**Error! Bookmark not defined.**
Bảng 4.3. Bảng độ trễ truyền dữ liệu 62
Bảng 4.4. Bảng so sánh tài nguyên hệ thống sử dụng 63

CHƯƠNG 1: TỔNG QUAN

1.1. Đặt vấn đề và động cơ nghiên cứu

Trong thời đại cách mạng công nghiệp 4.0, Internet vạn vật (Internet of Things – IoT) đang trở thành một trong những xu hướng công nghệ quan trọng nhất. IoT là sự kết nối giữa các thiết bị vật lý với nhau và với Internet, nhằm trao đổi dữ liệu, tự động hóa và hỗ trợ ra quyết định trong nhiều lĩnh vực như y tế, giao thông, nông nghiệp, công nghiệp, nhà thông minh và đô thị thông minh. Dự báo đến năm 2030, số lượng thiết bị IoT sẽ vượt mốc 25 tỷ thiết bị trên toàn cầu, cho thấy tốc độ phát triển cực kỳ nhanh chóng của công nghệ này.

Tuy nhiên, song hành với sự phát triển đó là những lo ngại ngày càng tăng về bảo mật và an toàn thông tin trong các hệ thống IoT. Khác với các hệ thống CNTT truyền thống, hệ thống IoT có đặc điểm là phân tán, tài nguyên hạn chế, và thường thiếu các cơ chế bảo mật mạnh mẽ do ưu tiên hiệu năng và chi phí. Điều này khiến IoT trở thành mục tiêu hấp dẫn cho tin tặc, dẫn đến hàng loạt các sự cố như:

- Tấn công Mirai Botnet làm tê liệt hàng loạt hệ thống bằng cách chiếm quyền điều khiển camera, router.
- Các thiết bị cảm biến bị giả mạo để gửi dữ liệu sai lệch, gây ảnh hưởng đến hệ thống điều khiển công nghiệp hoặc y tế.
- Thiết bị bị nghe lén hoặc kiểm soát từ xa do truyền dữ liệu không mã hóa.

Trong bối cảnh đó, bảo mật cho hệ thống IoT đã trở thành một thách thức cấp thiết. Đặc biệt, tầng giao tiếp (communication layer) – nơi các thiết bị IoT trao đổi dữ liệu với nhau thông qua các giao thức như MQTT, CoAP, AMQP – là một trong những điểm yếu thường bị khai thác đầu tiên bởi các cuộc tấn công. Trong đó, MQTT (Message Queuing Telemetry Transport) – giao thức phổ biến nhất trong các hệ thống IoT hiện nay – thường được triển khai với thiết lập mặc định không có mã hóa và xác thực, dẫn đến lỗ hổng nghiêm trọng.

Nhận thấy tầm quan trọng và tính thời sự của vấn đề, em quyết định thực hiện đề tài:

“Nghiên cứu nguy cơ mất an toàn trong IoT – Mô phỏng, đánh giá và đề xuất giải pháp bảo mật với MQTT” .

Mục tiêu chính của đề tài là mô phỏng một hệ thống IoT cơ bản sử dụng giao thức MQTT, thực hiện các kịch bản tấn công phổ biến như nghe lén (sniffing), giả mạo (spoofing), tấn công từ chối dịch vụ (DoS) và sau đó triển khai các cơ chế bảo mật như mã hóa (TLS), xác thực người dùng và phân quyền truy cập để bảo vệ hệ thống. Từ đó, đánh giá hiệu quả của các giải pháp đề xuất và rút ra bài học ứng dụng cho các hệ thống IoT thực tế.

Việc nghiên cứu này không chỉ giúp sinh viên nắm rõ hơn về nguyên lý hoạt động và các lỗ hổng bảo mật trong môi trường IoT, mà còn góp phần nâng cao nhận thức về bảo vệ dữ liệu trong thế giới kết nối ngày nay. Đồng thời, đề tài cũng hướng đến việc xây dựng một mô hình mô phỏng đơn giản, dễ tiếp cận nhưng có tính ứng dụng thực tiễn cao, có thể mở rộng trong tương lai cho các nghiên cứu sâu hơn về an ninh mạng trong hệ thống nhúng và điều khiển tự động.

1.2. Khảo sát thực trạng

Sự phát triển nhanh chóng của hệ sinh thái Internet of Things (IoT) đã tạo ra một mạng lưới rộng lớn các thiết bị thông minh được triển khai trong nhiều lĩnh vực như nhà thông minh, y tế, nông nghiệp, công nghiệp và đô thị thông minh. Tuy nhiên, thực tế hiện nay cho thấy bảo mật trong các hệ thống IoT vẫn chưa được chú trọng đúng mức, dẫn đến nhiều lỗ hổng nghiêm trọng có thể bị khai thác bởi các đối tượng tấn công mạng.

1.2.1. Bức tranh tổng thể về an ninh trong IoT

Theo các báo cáo từ những tổ chức an ninh mạng uy tín như Kaspersky, Palo Alto Networks hay OWASP (Open Web Application Security Project), phần lớn các thiết bị IoT hiện nay không được trang bị các cơ chế bảo mật mặc định, hoặc sử dụng các thiết lập bảo mật yếu như:

- Mặc định không có xác thực người dùng.
- Truyền dữ liệu qua giao thức không mã hóa (HTTP, MQTT không dùng TLS).

- Sử dụng tài khoản mặc định như admin/admin, không yêu cầu đổi mật khẩu ban đầu.
- Hệ điều hành thiết bị không được cập nhật vá lỗi thường xuyên.

Hệ quả là các thiết bị dễ dàng trở thành điểm yếu trong toàn bộ hệ thống, bị chiếm quyền điều khiển hoặc bị biến thành công cụ trong các cuộc tấn công lớn hơn.

1.2.2. Các cuộc tấn công thực tế từng xảy ra

Một số vụ tấn công mạng lớn sử dụng thiết bị IoT làm bàn đạp trong thời gian gần đây đã minh chứng rõ cho nguy cơ hiện hữu:

- **Cuộc tấn công Mirai (2016):** Một botnet khổng lồ được xây dựng từ các thiết bị IoT như camera IP, router, DVR... sử dụng mật khẩu mặc định. Botnet này đã phát động tấn công từ chối dịch vụ (DDoS) quy mô lớn khiến nhiều dịch vụ lớn như Twitter, Netflix, GitHub bị gián đoạn.
- **Tấn công vào hệ thống điều khiển từ xa tại một nhà máy nước ở Mỹ (2021):** Hacker xâm nhập vào hệ thống điều khiển và tăng nồng độ hóa chất trong nước đến mức nguy hiểm, cho thấy lỗ hổng trong việc xác thực và phân quyền trong các hệ thống IoT công nghiệp.
- **Sự cố Ring camera:** Các camera an ninh thông minh bị hacker xâm nhập do người dùng không đổi mật khẩu mặc định, dẫn đến việc nghe lén, giám sát trái phép trong các hộ gia đình.

Các sự cố này phản ánh thực trạng rằng mức độ bảo mật trong nhiều hệ thống IoT chưa tương xứng với rủi ro tiềm ẩn mà chúng mang lại.

1.2.3. Giao thức giao tiếp – điểm yếu phổ biến trong hệ thống IoT

Một trong những lỗ hổng trọng yếu trong hệ thống IoT đến từ tầng giao tiếp (Communication Layer) – nơi các thiết bị gửi và nhận dữ liệu. Trong đó, MQTT là một trong những giao thức phổ biến nhất hiện nay do đặc điểm nhẹ, dễ triển khai và tối ưu cho mạng có băng thông thấp. Tuy nhiên, MQTT:

- Mặc định không sử dụng mã hóa → dữ liệu có thể bị nghe lén trên mạng.
- Thiếu cơ chế xác thực mạnh → dễ bị giả mạo thiết bị.
- Không kiểm soát quyền truy cập rõ ràng → cho phép subscribe/publish không giới hạn nếu không được cấu hình đúng.

Nhiều hệ thống triển khai MQTT ở trạng thái mặc định, không bật TLS hoặc xác thực người dùng, dẫn đến nguy cơ bị sniffing, spoofing, replay attack, DoS attack rất cao.

1.2.4. Khảo sát hệ thống mô phỏng trong khuôn khổ đồ án

Để kiểm chứng và mô phỏng các nguy cơ trên, nhóm thực hiện đồ án đã xây dựng một mô hình hệ thống IoT đơn giản bao gồm:

- **Client (thiết bị IoT mô phỏng):** Gửi dữ liệu cảm biến nhiệt độ định kỳ.
- **MQTT Broker (máy chủ trung gian):** Nhận dữ liệu và chuyển tiếp tới các subscriber.
- **Attacker (kẻ tấn công):** Thực hiện các hành vi tấn công như nghe lén, giả mạo dữ liệu, hoặc flood để gây nghẽn dịch vụ.

Mô hình này cho phép nhóm khảo sát thực tế mức độ dễ bị tổn thương của hệ thống khi không áp dụng các biện pháp bảo mật cơ bản, đồng thời là cơ sở để triển khai các biện pháp khắc phục như mã hóa kênh truyền TLS, xác thực client, kiểm soát quyền truy cập topic, giúp nâng cao an toàn cho hệ thống IoT.

1.3. Đánh giá

Thông qua việc khảo sát thực trạng và phân tích các vấn đề bảo mật trong hệ thống IoT hiện nay, có thể đưa ra một số nhận định và đánh giá tổng quan như sau:

1.3.1. Ưu điểm của hệ thống IoT

Hệ thống IoT đã và đang mang lại nhiều lợi ích rõ rệt cho các cá nhân, doanh nghiệp và tổ chức trong nhiều lĩnh vực khác nhau. Những ưu điểm nổi bật bao gồm:

- **Tính tự động hóa cao**: Các thiết bị IoT có thể tự động thu thập và gửi dữ liệu theo thời gian thực, giảm thiểu sự can thiệp của con người trong các quy trình giám sát, điều khiển và vận hành**.**
- **Tiết kiệm chi phí và thời gian**: Việc thu thập dữ liệu từ xa, xử lý tức thời và điều khiển thiết bị nhanh chóng giúp tối ưu hóa hiệu suất và tiết kiệm chi phí vận hành.

- **Khả năng mở rộng và linh hoạt**: IoT cho phép dễ dàng mở rộng số lượng thiết bị hoặc chức năng trong hệ thống mà không ảnh hưởng đến các thành phần khác.
- **Tăng cường trải nghiệm người dùng**: Với các ứng dụng trong nhà thông minh, y tế, giao thông, người dùng có thể tương tác với môi trường sống một cách tiện lợi và cá nhân hóa hơn**.**

Tuy nhiên, đi kèm với sự phát triển nhanh chóng là hàng loạt vấn đề về bảo mật và an toàn dữ liệu mà nhiều tổ chức vẫn chưa kịp thích ứng hoặc khắc phục.

1.3.2. Hạn chế và thách thức về bảo mật trong IoT

Bên cạnh những ưu điểm nêu trên, hệ thống IoT hiện nay vẫn còn tồn tại nhiều hạn chế nghiêm trọng liên quan đến bảo mật, có thể kể đến:

- **Thiếu cơ chế bảo mật mặc định**: Nhiều thiết bị IoT khi xuất xưởng không có tính năng mã hóa, xác thực hoặc chỉ dùng tài khoản mặc định.
- **Tài nguyên hạn chế:** Các thiết bị IoT thường có cấu hình phần cứng yếu, dung lượng bộ nhớ nhỏ, dẫn đến khó tích hợp các giải pháp bảo mật hiện đại như mã hóa mạnh hoặc kiểm soát truy cập chi tiết.
- **Phân tán và khó kiểm soát:** Hệ thống IoT thường bao gồm nhiều thiết bị đặt ở nhiều vị trí khác nhau, khiến việc cập nhật, quản lý và giám sát trở nên khó khăn.
- **Giao thức không an toàn:** Các giao thức phổ biến trong IoT như MQTT, CoAP thường được sử dụng ở dạng cơ bản, không có mã hóa TLS hay xác thực người dùng, tạo điều kiện thuận lợi cho các cuộc tấn công như sniffing, spoofing, hoặc DoS.
- **Thiếu nhận thức bảo mật từ người dùng:** Nhiều người dùng cuối không đổi mật khẩu mặc định, không cập nhật firmware thiết bị, dẫn đến nguy cơ bị khai thác cao.
- **Thiếu nhận thức bảo mật từ người dùng:** Nhiều người dùng cuối không đổi mật khẩu mặc định, không cập nhật firmware thiết bị, dẫn đến nguy cơ bị khai thác cao.

1.3.1. Vai trò của bảo mật trong mô hình hệ thống IoT

Bảo mật không còn là một tùy chọn trong hệ thống IoT, mà là một yếu tố bắt buộc để đảm bảo sự tin cậy và an toàn của toàn bộ hệ sinh thái. Nếu không có các biện pháp bảo mật phù hợp:

- Hệ thống có thể bị gián đoạn hoạt động, gây hậu quả nghiêm trọng trong các lĩnh vực quan trọng như y tế, giao thông, công nghiệp.
- Dữ liệu cá nhân và nhạy cảm có thể bị rò rỉ, xâm phạm quyền riêng tư của người dùng.
- Hệ thống có thể bị chiếm quyền điều khiển và trở thành công cụ tấn công các hệ thống khác, gây thiệt hại lớn về tài chính và uy tín.

1.3.3. Định hướng nghiên cứu trong đồ án

Từ những đánh giá trên, đồ án tập trung vào việc nhận diện và mô phỏng các nguy cơ mất an toàn trong hệ thống IoT sử dụng giao thức MQTT, đồng thời đưa ra các giải pháp bảo mật cơ bản nhưng thiết thực để cải thiện tính an toàn của hệ thống.

Các nội dung nghiên cứu và thử nghiệm sẽ được thực hiện trong môi trường mô phỏng bằng VirtualBox, kết hợp các công cụ như Kali Linux, Alpine Linux, Mosquitto, Node-RED để tái hiện các tình huống tấn công thực tế và đề xuất hướng xử lý.

1.4. Nhiệm vụ đồ án

Dựa trên những đánh giá và thực trạng đã trình bày, nhiệm vụ chính của đồ án là nghiên cứu, mô phỏng và đánh giá các nguy cơ mất an toàn trong hệ thống IoT sử dụng giao thức MQTT – một giao thức phổ biến trong truyền thông M2M (Machine-to-Machine), đồng thời đề xuất và kiểm chứng các giải pháp bảo mật cơ bản nhưng hiệu quả. Cụ thể, các nhiệm vụ chính của đồ án bao gồm

1.4.1. Nghiên cứu cơ sở lý thuyết về IoT và bảo mật

- Tìm hiểu khái niệm, đặc điểm và kiến trúc tổng thể của hệ thống Internet of Things (IoT), bao gồm các mô hình 3 lớp (cảm biến – mạng – ứng dụng) và 5 lớp (nhận thức – mạng – xử lý – dịch vụ – ứng dụng).
- Phân tích cơ chế hoạt động, vai trò và cách triển khai của giao thức MQTT trong truyền thông giữa các thiết bị IoT.

- Khảo sát các điểm yếu và lỗ hổng bảo mật phổ biến trong hệ thống IoT, đặc biệt là với giao thức MQTT khi không có mã hóa hoặc xác thực.
- Tìm hiểu các phương pháp bảo mật phổ biến có thể áp dụng cho môi trường IoT như mã hóa TLS/SSL, xác thực người dùng, kiểm soát truy cập, và cập nhật phần mềm định kỳ.

1.4.2. Thiết kế mô hình mô phỏng hệ thống IoT

Thiết kế một mô hình mô phỏng hệ thống IoT với ba thành phần chính:

- **Client (thiết bị IoT):** Mô phỏng cảm biến nhiệt độ gửi dữ liệu định kỳ lên broker thông qua giao thức MQTT.
- **Broker (máy chủ trung gian):** Cài đặt Mosquitto làm broker nhận và phân phối dữ liệu MQTT.
- **Attacker (kẻ tấn công):** Sử dụng các công cụ tấn công như Wireshark, Kali Linux để thực hiện sniffing, chiếm quyền điều khiển, hoặc khai thác lỗ hổng xác thực.

Triển khai mô hình trong môi trường ảo hóa bằng VirtualBox với các máy ảo sử dụng hệ điều hành khác nhau như Alpine Linux, Ubuntu Server, Kali Linux.

1.4.3. Mô phỏng các cuộc tấn công điển hình vào hệ thống IoT

Xây dựng các kịch bản tấn công cụ thể vào hệ thống MQTT như:

- **Sniffing:** Theo dõi lưu lượng truyền dữ liệu không mã hóa để lấy cắp thông tin.
- **DoS/DDoS:** Làm gián đoạn hoạt động của broker bằng cách gửi lượng lớn yêu cầu kết nối giả mạo.
- **MQTT Injection:** Gửi nội dung giả mạo vào các chủ đề (topic) để đánh lừa các thiết bị IoT khác.
- **Brute-force/xác thực yếu:** Thử mật khẩu hoặc chiếm quyền truy cập do broker không yêu cầu xác thực hoặc chỉ sử dụng tài khoản mặc định.

1.4.4. Triển khai và kiểm nghiệm các giải pháp bảo mật

- Cấu hình xác thực người dùng trong Mosquitto để yêu cầu tên đăng nhập và mật khẩu khi client kết nối.
- Thiết lập mã hóa dữ liệu bằng giao thức TLS để đảm bảo tính bảo mật trong quá trình truyền tải dữ liệu giữa client và broker.

- Thử nghiệm lại các kịch bản tấn công đã thực hiện trong môi trường bảo mật để đánh giá mức độ hiệu quả của các giải pháp được áp dụng.

1.5. Đối tượng và phạm vi ứng dụng

1.5.1. Đối tượng ứng dụng

Đề tài được xây dựng nhằm phục vụ cho việc nghiên cứu, học tập và thực hành các vấn đề liên quan đến bảo mật trong hệ thống IoT, đặc biệt là trên môi trường mô phỏng. Các đối tượng cụ thể mà mô hình hướng đến bao gồm:

- Sinh viên, học viên ngành An ninh mạng, CNTT hoặc Điện tử - Viễn thông**:**
  Có thể sử dụng mô hình để hiểu rõ hơn về kiến trúc hệ thống IoT, giao thức MQTT, cũng như cách kẻ tấn công có thể khai thác các điểm yếu bảo mật phổ biến.
- Giảng viên, nhà nghiên cứu, chuyên gia kỹ thuật**:**
  Có thể tham khảo mô hình làm cơ sở để giảng dạy, hướng dẫn thực hành hoặc mở rộng thêm các kịch bản tấn công và giải pháp bảo mật phức tạp hơn**.**
- Doanh nghiệp hoặc tổ chức triển khai hệ thống IoT (ở quy mô nhỏ):
  Có thể lấy ý tưởng từ mô hình này để kiểm tra sơ bộ khả năng bảo mật cho các thiết bị cảm biến IoT sử dụng MQTT trước khi triển khai vào môi trường thực tế**.**

1.5.2. Phạm vi ứng dụng

Do đề tài thuộc phạm vi đồ án cơ sở ngành, hệ thống được thiết kế và triển khai trong môi trường mô phỏng với các giới hạn rõ ràng về chức năng và quy mô, cụ thể:

- **Phạm vi kỹ thuật:**
  + Mô phỏng một hệ thống IoT đơn giản gồm: thiết bị cảm biến (client), máy chủ trung gian MQTT (broker), và một máy tấn công (attacker).
  + Sử dụng giao thức MQTT – một giao thức nhẹ và phổ biến trong truyền thông IoT.
  + Các cuộc tấn công được thực hiện trong môi trường kiểm soát, không áp dụng trên thiết bị IoT thật hoặc hạ tầng mạng thực tế.
- **Phạm vi bảo mật:**

+ Chỉ tập trung vào các mối đe dọa phổ biến ở tầng giao thức MQTT, như: thiếu mã hóa, thiếu xác thực, tấn công giả mạo hoặc nghe lén dữ liệu.
+ Các giải pháp được áp dụng gồm: xác thực người dùng, mã hóa kết nối với TLS, giám sát và phát hiện truy cập bất thường.

- **Phạm vi công cụ:**
  + Sử dụng phần mềm mã nguồn mở và công cụ mô phỏng miễn phí như: VirtualBox, Kali Linux, Alpine Linux, Ubuntu Server, Mosquitto MQTT, Wireshark, Node-RED, Python.

1.6. Cấu trúc đồ án

Đồ án được chia thành 5 chương chính, được xây dựng theo trình tự logic từ tổng quan lý thuyết đến mô hình thực nghiệm và đánh giá kết quả. Cụ thể như sau:

Chương 1: TỔNG QUAN

Chương này giới thiệu tổng quan về đề tài nghiên cứu, lý do chọn đề tài, thực trạng hệ thống IoT trong bối cảnh hiện nay và các vấn đề mất an toàn thông tin phổ biến. Đồng thời, chương cũng trình bày nhiệm vụ nghiên cứu, đối tượng và phạm vi ứng dụng, cùng từ khóa chuyên ngành giúp định hướng nội dung đồ án.

Chương 2: CƠ SỞ LÝ THUYẾT

Chương này trình bày những kiến thức nền tảng liên quan đến đề tài, bao gồm:

- Kiến trúc hệ thống IoT và các lớp cấu trúc phổ biến (3 lớp, 5 lớp).
- Giao thức MQTT – giao thức truyền thông nhẹ dành cho IoT: cách hoạt động, ưu điểm, nhược điểm và các điểm yếu bảo mật.
- Các kiểu tấn công phổ biến vào hệ thống IoT như: giả mạo, nghe lén, tấn công từ chối dịch vụ, ...
- Các biện pháp bảo mật cơ bản có thể áp dụng để phòng chống các nguy cơ này.
- Giới thiệu các công cụ và phần mềm sử dụng trong mô phỏng.

- Chương này đóng vai trò nền tảng lý thuyết để triển khai phần thực nghiệm ở chương 3.

Chương 3: THIẾT KẾ HỆ THỐNG VÀ GIẢI PHÁP BẢO MẬT

Chương này trình bày chi tiết quá trình thiết kế và triển khai mô hình hệ thống mô phỏng IoT. Nội dung bao gồm:

- Kiến trúc tổng thể của hệ thống mô phỏng (client – broker – attacker).
- Thiết kế và cài đặt từng thành phần trong mô hình trên các máy ảo.
- Kịch bản tấn công sử dụng các công cụ mô phỏng trên Kali Linux.
- Các giải pháp bảo mật được áp dụng: xác thực client, mã hóa TLS, cấu hình broker an toàn.
- Cập nhật và triển khai mã nguồn cho thiết bị cảm biến (client).
- Đây là phần quan trọng nhất thể hiện khả năng ứng dụng lý thuyết vào thực tiễn.

Chương 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này ghi nhận kết quả đạt được sau khi triển khai mô hình, gồm:

- So sánh hệ thống IoT có áp dụng bảo mật và không áp dụng bảo mật.
- Đánh giá hiệu quả của các giải pháp bảo mật đã triển khai.
- Phân tích các chỉ số liên quan như độ trễ, mức tiêu thụ tài nguyên, tính ổn định kết nối và khả năng chống lại tấn công.
- Qua chương này, đồ án thể hiện được tính thực nghiệm và khả năng phân tích, đánh giá khách quan của sinh viên.

Chương 5: KẾT LUẬN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN

Chương cuối cùng tổng hợp lại những nội dung chính của đồ án, đánh giá kết quả đạt được, những hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai như:

- Tích hợp các biện pháp phát hiện xâm nhập (IDS).
- Mở rộng giao thức khác ngoài MQTT.
- Kết nối với thiết bị phần cứng thật (ESP32, Raspberry Pi, ...).

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về Internet of Things (IoT)

2.1.1. Khái niệm IoT

Internet of Things (IoT – Internet vạn vật) là một mô hình mạng cho phép các thiết bị vật lý (devices) như cảm biến, máy móc, thiết bị điện tử, phương tiện, vật dụng gia đình, ... kết nối với Internet để thu thập, gửi và nhận dữ liệu mà không cần sự can thiệp trực tiếp của con người. Những thiết bị này được trang bị bộ vi xử lý, mô-đun truyền thông, cảm biến và phần mềm để thực hiện quá trình trao đổi dữ liệu thông minh, liên tục.

IoT được ứng dụng rộng rãi trong các lĩnh vực như: nhà thông minh, nông nghiệp thông minh, công nghiệp 4.0, chăm sóc sức khỏe, giao thông thông minh, … Tuy nhiên, việc kết nối Internet giữa nhiều thiết bị cũng kéo theo những nguy cơ lớn về an toàn thông tin và bảo mật hệ thống.

2.1.2. Các thành phần cơ bản trong hệ thống IoT

Một hệ thống IoT thường bao gồm các thành phần chính sau:

- **Thiết bị đầu cuối (End devices):** Bao gồm các cảm biến (sensor), thiết bị truyền động (actuator) thu thập dữ liệu và thực hiện hành động.
- **Bộ điều khiển cục bộ (Gateway):** Là thiết bị trung gian giữa cảm biến và Internet. Có thể xử lý dữ liệu sơ bộ và định tuyến thông tin.
- **Hạ tầng mạng (Network):** Cung cấp kết nối (Wi-Fi, ZigBee, LTE, LoRaWAN...) giữa các thiết bị và server.
- **Máy chủ/broker:** Lưu trữ, xử lý và phân phối dữ liệu (thường dùng giao thức MQTT, HTTP hoặc CoAP).
- **Ứng dụng người dùng:** Giao diện quản lý dữ liệu (trình duyệt, ứng dụng, dashboard, ...).

2.1.3. Kiến trúc hệ thống IoT

a) Kiến trúc 3 lớp

Kiến trúc 3 lớp là mô hình cơ bản nhất, gồm ba tầng:

- **Lớp cảm nhận (Perception Layer):** Là tầng gần thiết bị nhất, chịu trách nhiệm thu thập dữ liệu vật lý từ môi trường thông qua cảm biến hoặc thiết bị RFID.
- **Lớp mạng (Network Layer):** Truyền dữ liệu thu thập được từ lớp cảm nhận đến trung tâm xử lý. Tầng này sử dụng các giao thức như MQTT, HTTP, CoAP thông qua mạng Wi-Fi, 4G/5G hoặc mạng riêng.
- **Lớp ứng dụng (Application Layer):** Cung cấp các dịch vụ IoT cho người dùng cuối (giám sát, cảnh báo, điều khiển, ...).

b) Kiến trúc 5 lớp

Kiến trúc 5 lớp mở rộng từ kiến trúc 3 lớp, thêm tính linh hoạt và bảo mật:

- **Lớp cảm nhận (Perception Layer):** Cảm biến, camera, RFID, ...
- **Lớp mạng (Network Layer):** Truyền dữ liệu đến hệ thống điện toán.
- **Lớp xử lý (Processing Layer):** Xử lý, lưu trữ dữ liệu, đưa ra quyết định tạm thời (sử dụng máy chủ, điện toán đám mây hoặc biên - edge computing).
- **Lớp dịch vụ (Service Layer):** Điều phối các dịch vụ IoT như phát hiện sự kiện, phản hồi hành động, ...
- **Lớp ứng dụng (Application Layer):** Giao diện tương tác người dùng, dashboard, cảnh báo, ...

So sánh:

Mô hình 5 lớp phù hợp với các hệ thống IoT lớn, có yêu cầu bảo mật và xử lý phức tạp hơn. Mô hình 3 lớp đơn giản hơn, thích hợp cho các hệ thống nhỏ, mô phỏng hoặc thử nghiệm.

2.1.4. Liên hệ với hệ thống mô phỏng trong đồ án

Trong phạm vi của đồ án này, nhóm đã xây dựng một hệ thống IoT mô phỏng gồm ba thực thể chính:

- **Client (Thiết bị IoT):** Là một thiết bị cảm biến giả lập (chạy bằng script Python) có nhiệm vụ gửi dữ liệu (như nhiệt độ) định kỳ thông qua giao thức MQTT đến máy chủ trung gian (Broker). Thiết bị này chạy trên máy ảo Alpine Linux. Đây là lớp cảm nhận trong kiến trúc IoT, thực hiện việc thu thập và gửi dữ liệu.

- **Broker (MQTT Server):** Là máy chủ trung gian chịu trách nhiệm nhận, phân phối dữ liệu từ thiết bị gửi (publisher) đến thiết bị nhận (subscriber). Sử dụng phần mềm Eclipse Mosquitto trên máy ảo Ubuntu Server để đóng vai trò MQTT Broker. Broker thực hiện chức năng của lớp mạng, và một phần xử lý trong hệ thống IoT.
- **Attacker (Kẻ tấn công):** Là thực thể được sử dụng để mô phỏng các cuộc tấn công mạng vào hệ thống IoT (sniffing, MITM, brute-force, ...). Máy này sử dụng Kali Linux, một bản phân phối mạnh mẽ cho kiểm thử xâm nhập và bảo mật.

Hệ thống mô phỏng này phản ánh một mô hình IoT thực tế với đầy đủ các thành phần chính:

- Lớp cảm nhận → Client IoT device (sensor giả lập).
- Lớp mạng → Broker MQTT truyền dữ liệu.
- Lớp ứng dụng → Giao diện Node-RED hoặc ghi log dữ liệu để theo dõi.
- Ngoài ra, thực thể Attacker đóng vai trò kiểm thử khả năng bảo mật và cho phép đánh giá các điểm yếu trong mô hình.

```
+------------------+          MQTT Publish
|   Client (Alpine)|-------------------------+
|  Sensor Emulator |                         |
+------------------+                         /
                                             v
                                     +--------------+
                                     |   Broker     |
                                     | Mosquitto    |
                                     | (Ubuntu)     |
                                     +--------------+
                                             ^
                                             |
+------------------+       MQTT Sniffing     /
|   Attacker       |<----------------------+
|  (Kali Linux)    |
+------------------+
```

Hình 2.1. Sơ đồ kiến trúc mô phỏng

2.2. Giao thức MQTT trong hệ thống IoT

2.2.1. Tổng quan về MQTT

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền thông nhẹ, dựa trên mô hình publish/subscribe (xuất bản/đăng ký), được thiết kế đặc biệt cho các thiết bị có tài nguyên hạn chế và mạng không ổn định. Giao thức này được phát triển lần đầu tiên bởi IBM vào năm 1999 nhằm phục vụ cho các hệ thống giám sát từ xa.

Với kiến trúc đơn giản, ít tốn băng thông và tài nguyên, MQTT đã trở thành một trong những giao thức phổ biến nhất trong hệ sinh thái Internet of Things (IoT). Nó hỗ trợ nhiều kiểu thiết bị từ cảm biến nhỏ gọn đến các hệ thống phức tạp như nhà máy công nghiệp, và có thể hoạt động hiệu quả trong môi trường có độ trễ cao, băng thông thấp hoặc mất kết nối thường xuyên.

2.2.2. Kiến trúc và cơ chế hoạt động của MQTT

a. Mô hình giao tiếp MQTT

MQTT hoạt động dựa trên kiến trúc client-broker, trong đó client không giao tiếp trực tiếp với nhau mà gửi/nhận dữ liệu thông qua một trung gian gọi là broker.

Các thành phần chính:

- Publisher (Thiết bị gửi dữ liệu): Gửi thông điệp (message) đến một "chủ đề" (topic).
- Subscriber (Thiết bị nhận dữ liệu): Đăng ký nhận thông điệp từ một hoặc nhiều topic.
- Broker (Máy chủ trung gian): Quản lý tất cả kết nối từ client, nhận message từ Publisher và chuyển tiếp đến các Subscriber tương ứng.

b. Sơ đồ mô hình hoạt động

```
[Publisher] ---> gửi dữ liệu ---> [Broker] ---> chuyển đến ---> [Subscriber]
                  (topic: iot/temp)                  (topic khớp: iot/#)
```

Hình 2.2. Sơ đồ mô hình hoạt động

c. Luồng hoạt động tiêu chuẩn

- Client kết nối tới Broker thông qua TCP (port mặc định: 1883 hoặc 8883 với SSL).
- Publisher gửi thông điệp tới một Topic cụ thể (ví dụ: iot/temperature).
- Broker nhận thông điệp và kiểm tra xem những client nào đã subscribe topic đó.
- Broker gửi thông điệp đến tất cả Subscriber phù hợp.

d. Các mức QoS (Quality of Service)

MQTT hỗ trợ ba mức QoS nhằm cân bằng giữa hiệu suất và độ tin cậy:

Mức QoS	Mô tả	Đảm bảo	Rủi ro
0	Gửi tối đa một lần (at most once)	Không xác nhận	Có thể mất dữ liệu
1	Gửi ít nhất một lần (at least once)	Có xác nhận	Có thể trùng lặp dữ liệu
2	Gửi chính xác một lần (exactly once)	Giao tiếp nhiều bước	Độ tin cậy cao, hiệu suất thấp

Bảng 2.1. Các mức QoS

2.2.3. Ứng dụng thực tiễn của MQTT trong hệ thống IoT

a. Các lĩnh vực áp dụng

- **Nhà thông minh (Smart Home):** Tự động hóa bật/tắt thiết bị, theo dõi trạng thái.
- **Công nghiệp 4.0:** Giám sát dây chuyền sản xuất theo thời gian thực.
- **Y tế từ xa (Telemedicine):** Truyền dữ liệu cảm biến từ bệnh nhân về trung tâm.
- **Nông nghiệp thông minh:** Cảm biến nhiệt độ, độ ẩm giám sát môi trường cây trồng.

b. Liên hệ với hệ thống mô phỏng của đồ án

Thành phần	Vai trò trong mô hình MQTT
iot-device	Là Publisher – thiết bị giả lập gửi dữ liệu cảm biến
Mosquitto broker	Là Broker trung gian xử lý mọi gói tin MQTT
Node-RED	Là Subscriber hiển thị dữ liệu trực quan hoặc xử lý logic
attacker (Kali)	Là client bất hợp pháp, có thể tấn công sniff, DoS, MITM

Bảng 2.2. Các thành phần trong MQTT

Hệ thống được triển khai trong môi trường ảo hóa gồm nhiều máy ảo (VirtualBox) để mô phỏng một hệ thống IoT thực tế, có thể quan sát các luồng dữ liệu MQTT trong Wireshark hoặc thực hiện các bài thử tấn công.

2.2.4. Các điểm yếu bảo mật của MQTT

Mặc dù MQTT rất phổ biến và hiệu quả, nhưng nó không được thiết kế với các yếu tố bảo mật cốt lõi từ đầu. Một số điểm yếu bảo mật nguy hiểm gồm:

a. Dữ liệu truyền dưới dạng rõ (plaintext)

- MQTT mặc định truyền qua cổng TCP không mã hóa (1883).
- Kẻ tấn công có thể sử dụng công cụ như Wireshark để nghe lén (sniffing) thông tin: dữ liệu cảm biến, chủ đề, mật khẩu người dùng nếu có.
- Không có mã hóa → dễ bị thu thập hoặc chỉnh sửa nội dung gói tin.

Tình huống mô phỏng: attacker chạy Wireshark trên máy cùng mạng, dễ dàng bắt được gói tin "topic": "iot/temp", "message": "25.6".

b. Thiếu xác thực client

- MQTT không yêu cầu username/password bắt buộc.
- Bất kỳ thiết bị nào cũng có thể kết nối tới broker nếu broker không giới hạn.
- Dẫn đến nguy cơ:
  + Giả mạo client để gửi dữ liệu sai lệch.
  + Tấn công từ chối dịch vụ (DoS) bằng kết nối hàng loạt.

c. Không kiểm soát quyền truy cập chủ đề (topic)

- Nếu broker không được cấu hình kiểm soát, bất kỳ client nào cũng có thể:
- Subscribe vào tất cả topic (#) → rò rỉ dữ liệu.
- Publish vào bất kỳ topic nào → inject dữ liệu giả.

d. Không xác minh danh tính broker

- Client không xác thực broker có đáng tin cậy không.
- Dễ bị tấn công Man-In-The-Middle (MITM) nếu attacker giả làm broker trung gian.

e. Dễ bị tấn công DoS và spam topic

- Không có giới hạn bảo vệ băng thông, attacker có thể:
- Gửi hàng loạt message rác vào broker.
- Tạo quá nhiều kết nối dẫn đến crash hoặc nghẽn mạng.

2.2.5. Biện pháp tăng cường bảo mật cho MQTT

Rủi ro bảo mật	Giải pháp đề xuất
Plaintext communication	Sử dụng giao thức TLS (MQTT over SSL, port 8883)
Không xác thực người dùng	Thiết lập username/password cho client khi kết nối broker
Không phân quyền	Cấu hình phân quyền bằng ACL (Access Control List) trong broker
Không kiểm soát broker	Dùng chứng chỉ SSL để xác minh danh tính broker (CA signed cert)
Tấn công DoS	Giới hạn số kết nối/giới hạn tần suất publish từ client
Không log/giám sát	Bật tính năng ghi log & cảnh báo bất thường trên Mosquitto

Bảng 2.3. Bảng giải pháp đề xuất

2.2.6. Đánh giá tổng quan

MQTT là một giao thức nhẹ, hiệu quả, nhưng thiếu bảo mật mặc định, đặc biệt nguy hiểm trong môi trường IoT vốn không được bảo vệ kỹ.

Trong môi trường mô phỏng của đồ án, MQTT thể hiện rõ cách dữ liệu có thể bị rò rỉ hoặc bị giả mạo nếu không được cấu hình đúng cách.

Phần này đóng vai trò làm nền tảng để thực hiện các kịch bản tấn công và phòng thủ trong chương tiếp theo.

2.3. Các loại tấn công phổ biến trên IoT

Hệ sinh thái IoT bao gồm hàng tỷ thiết bị được kết nối và truyền thông qua mạng, tạo thành một không gian phong phú nhưng cũng đầy rẫy các lỗ hổng bảo mật. Với đặc tính tài nguyên hạn chế, thiếu tiêu chuẩn bảo mật chung và mức độ triển khai đại trà, các thiết bị IoT trở thành mục tiêu hấp dẫn của nhiều loại tấn công mạng.

2.3.1. Tổng quan về nguy cơ mất an toàn trong IoT

Trong bối cảnh các hệ thống IoT ngày càng được triển khai rộng rãi ở nhiều lĩnh vực khác nhau như y tế, công nghiệp, giao thông, và nhà thông minh, các mối đe dọa về an ninh mạng đối với hệ thống này cũng gia tăng đáng kể. Một trong những nguyên nhân chính là do bản chất phân tán, tài nguyên hạn chế, và cấu trúc mở của các thiết bị IoT, khiến chúng trở thành mục tiêu dễ bị tấn công bởi các tác nhân độc hại.

Trong khuôn khổ đồ án này, nhóm nghiên cứu tập trung vào việc phân tích, mô phỏng các mô hình tấn công phổ biến trong môi trường IoT sử dụng giao thức MQTT – một giao thức truyền thông nhẹ và phổ biến trong lĩnh vực IoT. Việc thực hiện mô phỏng các loại tấn công cụ thể trong hệ thống ảo hóa giúp xác định mức độ rủi ro và đưa ra các đề xuất tăng cường bảo mật phù hợp.

Các nguy cơ thường gặp bao gồm:

- Rò rỉ dữ liệu do truyền không mã hóa.
- Tấn công giả mạo thiết bị.
- Tấn công từ chối dịch vụ (DoS).
- Chiếm quyền điều khiển thiết bị.
- Tấn công trung gian (MITM).

2.3.2. Tấn công nghe lén (Eavesdropping)

Đây là hình thức tấn công thụ động, trong đó kẻ tấn công lặng lẽ theo dõi và thu thập dữ liệu được truyền giữa các thiết bị IoT. Tấn công nghe lén là hành vi khai thác dữ liệu đang truyền trong mạng mà không cần can thiệp trực tiếp vào luồng giao tiếp. Đối với các thiết bị IoT sử dụng giao thức truyền tải không mã hóa như MQTT ở chế độ mặc định, các gói dữ liệu có thể bị bên thứ ba chặn lại và đọc nội dung dễ dàng thông qua các công cụ như Wireshark, tcpdump,...

- **Cơ chế:** Khi dữ liệu được truyền mà không mã hóa (plaintext), attacker có thể sử dụng các công cụ như Wireshark để chặn và đọc thông tin như nhiệt độ, vị trí, tín hiệu điều khiển…
- **Nguy hiểm:** Dữ liệu có thể bị thu thập để phân tích hành vi, tạo bản đồ hệ thống hoặc chuẩn bị cho các cuộc tấn công phức tạp hơn.
- **Ví dụ trong mô phỏng:** Dữ liệu MQTT truyền giữa client và broker bị bắt gói để xem nội dung tin nhắn, chủ đề (topic), thông điệp lệnh.

2.3.3. Tấn công giả mạo trung gian (Man-in-the-Middle – MITM)

Tấn công MITM xảy ra khi kẻ tấn công chèn mình vào giữa hai thiết bị đang giao tiếp để lén theo dõi, chỉnh sửa hoặc làm sai lệch dữ liệu được truyền. Trong môi trường IoT, MITM là một trong những kiểu tấn công nguy hiểm nhất vì các thiết bị thường không xác thực lẫn nhau và giao tiếp qua kênh không an toàn.

- **Cách thực hiện:** Dùng công cụ như ARP spoofing, DNS poisoning để điều hướng lưu lượng mạng về attacker trước khi truyền tiếp.
- **Mục tiêu:** Thay đổi nội dung tin nhắn MQTT, chèn lệnh điều khiển giả mạo hoặc đánh cắp thông tin nhạy cảm.

2.3.4. Tấn công từ chối dịch vụ (DoS – Denial of Service)

Tấn công DoS nhắm đến mục tiêu làm tê liệt dịch vụ bằng cách gửi lượng lớn yêu cầu không hợp lệ hoặc quá tải hệ thống. Trong IoT, do nhiều thiết bị có năng lực xử lý thấp và tài nguyên hạn chế, chỉ một lượng nhỏ yêu cầu độc hại cũng có thể gây sập hệ thống.

Một số hình thức DoS phổ biến:

- Gửi hàng loạt yêu cầu kết nối đến broker MQTT khiến broker không thể phục vụ thiết bị thật.
- Flooding gói dữ liệu liên tục đến một thiết bị cảm biến khiến nó ngừng phản hồi hoặc treo.

Đây là một trong những hình thức tấn công phổ biến và nguy hiểm nhất với hệ thống IoT.

- **Mục tiêu:** Làm cho hệ thống hoặc thiết bị không thể hoạt động bình thường bằng cách làm quá tải tài nguyên (CPU, RAM, băng thông).
- **Phương pháp:** Gửi hàng loạt kết nối hoặc gói tin đến MQTT Broker (hoặc API của thiết bị) làm hệ thống bị tê liệt. Làm mất kết nối giữa client và broker, hệ thống không thể ghi nhận hoặc xử lý dữ liệu đúng thời điểm.
- **Biến thể:** Nếu nhiều thiết bị IoT bị nhiễm mã độc, chúng có thể trở thành một mạng botnet (ví dụ: Mirai) để tấn công DDoS quy mô lớn.
- **Mô phỏng thực tế:** Dùng script Python để tạo hàng trăm kết nối giả đến Mosquitto Broker gây treo hoặc lag hệ thống.

2.3.5. Tấn công brute-force (Đoán mật khẩu)

Do nhiều hệ thống IoT có thiết lập mặc định yếu (ví dụ username/password: admin/admin), attacker có thể sử dụng kỹ thuật brute-force để thử hàng loạt tài khoản và mật khẩu để chiếm quyền điều khiển hệ thống.

Brute-force thường sử dụng kết hợp với công cụ như Hydra, Medusa, ... để tự động hóa quá trình thử mật khẩu và xác thực đến MQTT broker hoặc các giao diện quản lý thiết bị.

2.3.6. Tấn công phát lại (Replay Attack)

Attacker ghi lại dữ liệu đã truyền (ví dụ: tín hiệu mở khóa cửa thông minh), sau đó phát lại y hệt gói tin đó để đánh lừa thiết bị.

- **Điều kiện:** Thiết bị không có cơ chế xác thực thời gian thực, không kiểm tra tính duy nhất của gói tin.
- **Tác động:** Thực hiện các lệnh điều khiển mà không cần quyền truy cập thực sự.
- **Ví dụ:** Dùng script phát lại gói MQTT bật công tắc hoặc gửi dữ liệu cảm biến sai lệch.

2.3.7. Tấn công khai thác lỗ hổng phần mềm (Firmware Exploit)

Do nhiều thiết bị IoT sử dụng hệ điều hành hoặc firmware đơn giản, lỗi thời, nên dễ bị khai thác qua các lỗ hổng bảo mật đã được công bố (CVE).

- **Hình thức:** Attacker khai thác lỗi tràn bộ đệm, injection, hoặc dịch vụ mở không được bảo vệ để chiếm quyền điều khiển thiết bị.
- **Ví dụ thực tế:** Một số camera IP bị chiếm quyền qua giao diện quản trị web không có mật khẩu hoặc có mật khẩu mặc định.

2.3.8. Phân tích hành vi (Traffic Analysis)

Dù dữ liệu đã được mã hóa, attacker vẫn có thể phân tích mẫu lưu lượng để suy đoán các hành vi nhất định.

- **Ví dụ:** Nếu cứ 10 phút lại có dữ liệu gửi đi từ cảm biến, attacker có thể đoán được hệ thống đang giám sát theo chu kỳ, từ đó dự đoán thời gian hoạt động.

2.3.9. Tổng hợp và phân loại mức độ nguy hiểm

Loại tấn công	Tính chất	Mức độ nguy hiểm	Đặc biệt dễ xảy ra khi
Nghe lén	Thụ động	Trung bình	Không có mã hóa
Giả mạo	Chủ động	Cao	Thiếu xác thực
DoS/DDoS	Chủ động	Rất cao	Không giới hạn kết nối
Replay	Chủ động	Cao	Không kiểm tra tính duy nhất
MITM	Chủ động	Rất cao	Không bảo vệ lớp mạng
Firmware exploit	Chủ động	Rất cao	Phần mềm lỗi thời
Phân tích hành vi	Thụ động	Thấp –Trung bình	Không có bảo vệ lưu lượng

Bảng 2.4. Mức độ nguy hiểm của các loại tấn công trong IoT

2.4. Giới thiệu các biện pháp bảo mật cơ bản

Trong bối cảnh các hệ thống IoT ngày càng phổ biến, việc bảo đảm an toàn thông tin cho các thiết bị kết nối mạng là một thách thức quan trọng. Đặc biệt, với các hệ thống sử dụng giao thức MQTT – vốn được thiết kế với mục tiêu đơn giản và nhẹ – thì các vấn đề về bảo mật càng trở nên cấp thiết. Trong phần này, chúng ta sẽ đi sâu vào các biện pháp bảo mật cơ bản có thể áp dụng nhằm tăng cường mức độ an toàn cho hệ thống IoT sử dụng MQTT, bao gồm xác thực người dùng, mã hóa dữ liệu, phân quyền truy cập, và giám sát hoạt động mạng.

2.4.1. Xác thực người dùng (Authentication)

Mục tiêu: Đảm bảo chỉ các thiết bị hoặc người dùng hợp lệ mới được phép kết nối và truyền dữ liệu qua broker MQTT.

- **Username/Password:** Là cơ chế xác thực cơ bản nhất trong MQTT. Thông thường, khi client kết nối đến broker, nó sẽ gửi cặp username và password trong gói kết nối (CONNECT). Broker sẽ kiểm tra thông tin này với dữ liệu đã cấu hình sẵn (thường trong file password_file).
- **Nhược điểm:** Nếu không đi kèm với mã hóa, thông tin xác thực có thể bị lộ qua mạng trong dạng plaintext.
- **Biện pháp nâng cao:**
  + Sử dụng thư viện xác thực như mosquitto_passwd để mã hóa mật khẩu (SHA512).

+ Tích hợp xác thực OAuth2 hoặc qua hệ thống quản lý người dùng trung tâm (LDAP).

2.4.2. Mã hóa dữ liệu (Encryption)

Mục tiêu: Bảo vệ tính bảo mật và toàn vẹn của dữ liệu khi truyền qua mạng.

- **TLS/SSL:** MQTT hỗ trợ truyền dữ liệu qua kết nối TLS, một giao thức bảo mật cấp truyền tải được thiết kế để ngăn chặn việc nghe lén và giả mạo dữ liệu.
- **Cách triển khai:**
  + Cài đặt chứng chỉ tự ký hoặc từ CA (Certificate Authority) cho broker và client.
  + Bắt buộc client chỉ kết nối qua port bảo mật (ví dụ: 8883 thay vì 1883).
- **Lợi ích:**
  + Ngăn chặn sniffing (nghe lén).
  + Giảm rủi ro MITM (Man-in-the-Middle).

Lưu ý: Việc sử dụng mã hóa sẽ làm tăng độ trễ và tiêu tốn tài nguyên hơn, cần cân nhắc trong môi trường IoT giới hạn.

2.4.3. Phân quyền truy cập (Access Control)

Mục tiêu: Hạn chế quyền của từng thiết bị hoặc người dùng, chỉ cho phép truy cập vào các topic đã được phân quyền.

- **Topic ACL (Access Control List):** Broker như Mosquitto hỗ trợ ACL, cho phép chỉ định người dùng nào có quyền publish hoặc subscribe đến các topic nhất định.
- **Ví dụ:**
  + Người dùng A chỉ được phép publish lên sensor/temp.
  + Người dùng B chỉ được phép subscribe từ alert/#.
- **Ưu điểm:**
  + Tăng cường kiểm soát truy cập.
  + Hạn chế thiệt hại khi một node bị xâm nhập.

2.4.4. Giám sát và ghi log (Monitoring & Logging)

Mục tiêu: Phát hiện sớm các hoạt động bất thường và điều tra sự cố.

- **Broker log**: Theo dõi lịch sử kết nối, hành vi bất thường (spam message, brute-force).
- **Công cụ phân tích mạng**: Wireshark, tcpdump để kiểm tra lưu lượng.
- **IDS/IPS**: Tích hợp hệ thống phát hiện và ngăn chặn xâm nhập như Snort hoặc Suricata để bảo vệ tầng mạng.

2.4.1. Các biện pháp bổ sung

- **Rate limiting:** Giới hạn số lượng kết nối hoặc publish/subcribe mỗi đơn vị thời gian để ngăn DoS.
- **Cập nhật firmware định kỳ:** Đảm bảo thiết bị luôn sử dụng phần mềm an toàn mới nhất.
- **Cấu hình bảo mật trên broker:**
  + Tắt anonymous access (allow_anonymous false).
  + Không để lộ thông tin hệ thống qua lỗi hoặc thông báo.

2.5. Tổng kết

Các biện pháp bảo mật cơ bản tuy không thể đảm bảo tuyệt đối an toàn cho hệ thống IoT sử dụng MQTT, nhưng đóng vai trò nền tảng để ngăn chặn các tấn công phổ biến, đồng thời tạo cơ sở để triển khai các giải pháp bảo mật nâng cao hơn trong tương lai. Khi xây dựng một hệ thống thực tế, việc kết hợp nhiều cơ chế (defense-in-depth) là điều cần thiết để đảm bảo tính toàn vẹn, bảo mật và sẵn sàng của hệ thống.

2.6. Công cụ và phần mềm sử dụng trong mô phỏng

Để xây dựng một hệ thống mô phỏng có thể phản ánh rõ nét các nguy cơ mất an toàn thông tin trong môi trường IoT sử dụng giao thức MQTT, việc lựa chọn các công cụ và phần mềm phù hợp là yếu tố then chốt. Trong phần này, chúng tôi sẽ trình bày chi tiết các công cụ đã được sử dụng trong quá trình triển khai hệ thống mô phỏng, bao gồm hệ điều hành, nền tảng ảo hóa, phần mềm trung gian (middleware), công cụ tấn công – giám sát, cũng như các tiện ích hỗ trợ.

2.6.1. Oracel VirtualBox

Giới thiệu:

VirtualBox là một phần mềm ảo hóa mã nguồn mở do Oracle phát triển, cho phép người dùng tạo và quản lý nhiều máy ảo (VM) chạy đồng thời trên một hệ điều hành chủ (host). Đây là công cụ mạnh mẽ và linh hoạt, rất phù hợp trong việc xây dựng các môi trường thí nghiệm phân lập.

Chức năng:

VirtualBox được sử dụng để tạo và quản lý các máy ảo đóng vai trò khác nhau trong hệ thống mô phỏng:

- **Máy client IoT:** Giả lập thiết bị cảm biến gửi dữ liệu thông qua giao thức MQTT.
- **Broker MQTT:** Là máy trung gian xử lý và phân phối dữ liệu MQTT giữa các client.
- **Máy tấn công (attacker):** Giả lập kẻ tấn công thực hiện sniffing, MITM hoặc exploit lỗ hổng.

Lý do lựa chọn:

- Giao diện dễ sử dụng, cấu hình đơn giản.
- Hỗ trợ snapshot giúp lưu trạng thái và khôi phục dễ dàng.
- Cho phép tạo nhiều mô hình mạng ảo khác nhau (NAT, Host-only, Internal) thuận tiện cho mô phỏng các kịch bản bảo mật**.**

Miễn phí và hỗ trợ hầu hết các hệ điều hành phổ biến (Windows, Linux, macOS).

2.6.2. Alpine Linux

Giới thiệu:

Alpine Linux là một bản phân phối Linux siêu nhẹ, bảo mật cao, thường được sử dụng trong môi trường ảo hóa và container. Nhờ vào kích thước nhỏ gọn và mức tiêu thụ tài nguyên thấp, Alpine rất thích hợp để triển khai trong các hệ thống nhúng hoặc máy ảo nhẹ.

Chức năng:

- Dùng làm hệ điều hành chính cho các máy ảo đóng vai trò thiết bị IoT (publisher).
- Cài đặt các script Python cảm biến giả lập và giao tiếp MQTT với broker.

Lý do lựa chọn:

- Kích thước nhỏ (< 130MB), tốc độ khởi động nhanh, phù hợp máy ảo cấu hình thấp.
- Hệ thống bảo mật mặc định cao, tối giản các dịch vụ không cần thiết.
- Dễ cấu hình để phục vụ mục đích chuyên biệt như mô phỏng thiết bị đầu cuối IoT.

2.6.3. Kali Linux

Giới thiệu:

Kali Linux là hệ điều hành chuyên dụng dành cho kiểm thử xâm nhập (penetration testing) và đánh giá bảo mật hệ thống, được phát triển bởi Offensive Security. Hệ điều hành này tích hợp sẵn hàng trăm công cụ tấn công, phân tích lỗ hổng và giám sát mạng.

Chức năng:

- Là môi trường chính cho máy tấn công (attacker).
- Sử dụng để thực hiện các kỹ thuật sniffing, tấn công MITM, khai thác MQTT không mã hóa, ...
- Phân tích gói tin và kiểm tra tính bảo mật của hệ thống giao tiếp IoT.

Lý do lựa chọn:

- Hệ điều hành tiêu chuẩn trong kiểm thử an ninh mạng.
- Tích hợp sẵn công cụ như Wireshark, Nmap, Mosquitto client, ...
- Cộng đồng hỗ trợ mạnh, dễ học và mở rộng.

2.6.4. Eclipse Mosquitto (MQTT Broker)

Giới thiệu:

Mosquitto là một trình môi giới MQTT (MQTT Broker) mã nguồn mở được phát triển bởi Eclipse Foundation. Nó là một trong những triển khai MQTT phổ biến nhất hiện nay, hỗ trợ đầy đủ các phiên bản MQTT 3.1, 3.1.1 và 5.0.

Chức năng:

- Nhận và chuyển tiếp các gói tin MQTT giữa các thiết bị IoT.
- Cấu hình để hỗ trợ SSL/TLS, xác thực người dùng nhằm tăng cường bảo mật.
- Ghi log các hoạt động để phân tích luồng dữ liệu và hành vi hệ thống.

Lý do lựa chọn:

- Nhẹ, hiệu suất cao, cài đặt nhanh chóng trên nhiều hệ điều hành.
- Tài liệu hướng dẫn đầy đủ, hỗ trợ xác thực và mã hóa dữ liệu.
- Phù hợp với môi trường sản xuất lẫn mô phỏng nhỏ

2.6.5. Node-RED

Giới thiệu:

Node-RED là một công cụ lập trình trực quan do IBM phát triển, cho phép thiết kế luồng xử lý dữ liệu bằng giao diện kéo-thả, rất phổ biến trong các dự án IoT và tích hợp hệ thống.

Chức năng:

- Hiển thị trực quan dữ liệu cảm biến từ MQTT.
- Mô phỏng các phản hồi dựa trên dữ liệu IoT gửi về.
- Kiểm tra và theo dõi thời gian thực luồng dữ liệu giữa thiết bị và broker.

Lý do lựa chọn:

- Giao diện đồ họa dễ dùng, không cần lập trình phức tạp.
- Tích hợp nhanh với MQTT, REST API và hệ thống web.
- Có thể mở rộng dễ dàng thông qua thư viện các node tùy chỉnh.

2.6.6. Python + Paho MQTT

Giới thiệu:

Python là ngôn ngữ lập trình linh hoạt, phổ biến trong lĩnh vực IoT, tự động hóa và học máy. Thư viện Paho-MQTT (thuộc Eclipse) hỗ trợ giao tiếp MQTT dễ dàng từ các ứng dụng Python.

Chức năng:

- Tạo các script cảm biến IoT mô phỏng việc gửi dữ liệu lên broker.
- Gửi và nhận dữ liệu MQTT bằng Python thông qua thư viện Paho.
- Kiểm thử xác thực MQTT, mã hóa SSL/TLS và lỗi khi thiếu bảo mật.

Lý do lựa chọn:

- Python dễ học, cú pháp ngắn gọn, chạy ổn định trên môi trường Linux nhẹ.
- Paho-MQTT hỗ trợ đầy đủ các thao tác kết nối, publish, subscribe.
- Có thể nhanh chóng mở rộng mô hình với nhiều cảm biến hoặc xử lý logic nâng cao.

CHƯƠNG 3: CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể hệ thống mô phỏng

3.1.1. Mục tiêu thiết kế hệ thống

Mục tiêu chính của hệ thống mô phỏng là xây dựng một môi trường thử nghiệm hoàn chỉnh, trong đó có thể:

- Mô phỏng hoạt động giao tiếp của các thiết bị IoT sử dụng giao thức MQTT.
- Tái hiện các tình huống tấn công mạng phổ biến trong hệ sinh thái IoT như sniffing, MITM, khai thác không mã hóa và tấn công giả mạo.
- Thử nghiệm các giải pháp bảo mật như mã hóa SSL/TLS, xác thực client để đánh giá hiệu quả bảo vệ thông tin truyền tải.

Hệ thống cần đảm bảo tính độc lập giữa các thành phần, dễ cấu hình lại, và có thể mở rộng để kiểm thử nhiều tình huống khác nhau.

3.1.2. Các thành phần chính trong hệ thống

Hệ thống được thiết kế dựa trên mô hình **Client – Broker – Attacker**, gồm ba loại máy ảo chính:

a) Máy client IoT (Publisher/Subscriber)

- **Hệ điều hành**: Alpine Linux
- **Vai trò:** Mô phỏng thiết bị IoT (cảm biến nhiệt độ) sử dụng Python và thư viện Paho-MQTT để gửi dữ liệu định kỳ đến broker. Ngoài ra, cũng có thể đóng vai subscriber để nhận phản hồi.
- **Tính năng:**
  + Gửi dữ liệu môi trường (nhiệt độ, độ ẩm, ...) thông qua kênh MQTT.
  + Kết nối qua cổng 1883 (non-secure) hoặc 8883 (SSL/TLS).
  + Có thể được bảo vệ bằng xác thực MQTT và mã hóa.

b) Máy MQTT Broker (Mosquitto)

- **Hệ điều hành:** Ubuntu Server
- **Phần mềm:** Mosquitto Broker
- **Vai trò:** Là trung gian tiếp nhận dữ liệu từ các thiết bị client và phân phối đến các subscriber.
- **Tính năng:**

+ Hỗ trợ giao tiếp MQTT 3.1.1.
+ Có thể cấu hình xác thực người dùng, SSL/TLS.
+ Lưu log toàn bộ hoạt động publish/subscribe.

c) Máy attacker (Kẻ tấn công)

- **Hệ điều hành:** Kali Linux
- **Vai trò:** Thực hiện các cuộc tấn công vào hệ thống không bảo mật.
- **Công cụ sử dụng:** Wireshark (sniffing), Ettercap (MITM), Nmap (quét cổng), các script khai thác lỗ hổng.
- **Tính năng:**
  + Giám sát lưu lượng mạng khi MQTT không mã hóa.
  + Tấn công giả mạo để chiếm quyền kết nối hoặc gửi dữ liệu giả.

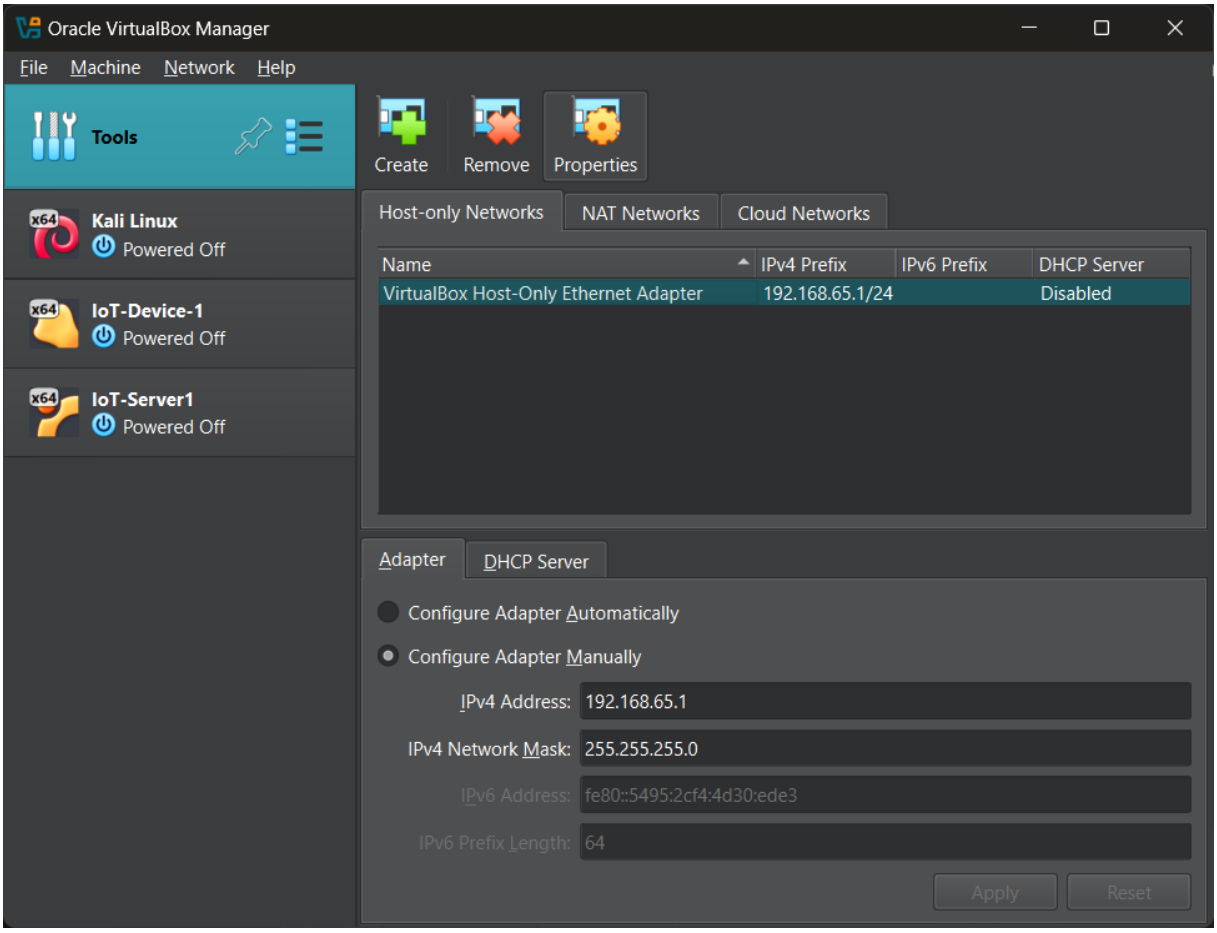
3.1.3. Mô hình mạng triển khai

Để đảm bảo mô phỏng an toàn và chính xác, mô hình mạng được cấu hình theo dạng Internal Network trong VirtualBox, giúp các máy ảo chỉ giao tiếp nội bộ với nhau mà không truy cập internet bên ngoài.

- Tên mạng nội bộ: VirtualBox Host-only
- IP tĩnh cấu hình thủ công cho từng máy:
  + Client IoT: 192.168.65.20
  + MQTT Broker: 192.168.65.10
  + Attacker: 192.168.65.100

Ưu điểm của cấu hình này:

- Tách biệt hoàn toàn với mạng thật, tránh rò rỉ gói tin.
- Cho phép attacker dễ dàng thực hiện sniffing, MITM trong môi trường khép kín.
- Dễ dàng kiểm soát lưu lượng để phân tích hành vi.



Hình 3.1. Cấu hình mạng nội bộ cho máy ảo

3.1.4. Sơ đồ kiến trúc tổng thể hệ thống

Dưới đây là sơ đồ minh họa kiến trúc hệ thống mô phỏng:

```
+--------------------+        +-----------------+        +-----------------+
|  IoT Device (Client)| --->  | MQTT Broker     | --->   | Node-RED / Logs |
| Alpine Linux       |        | Ubuntu Server   |        | Ubuntu Server   |
+--------------------+        +-----------------+        +-----------------+
          ↑                           ↑                          ↑
          |                           |                          |
      (Sniffing)                   (MITM)                  (Brute-force)
          ↑                           ↑                          ↑
+--------------------+        +-------------------------+
|    Attacker (MITM) | <------| Kali Linux VM           |
+--------------------+        +-------------------------+
```

Hình 3.2. Sơ đồ minh họa hệ thống

3.1.5. Tổng kết

Với mô hình thiết kế gồm ba máy ảo đại diện cho ba thực thể quan trọng trong hệ thống IoT – thiết bị đầu cuối, máy chủ trung gian và kẻ tấn công – kiến trúc hệ thống cho phép dễ dàng triển khai các kịch bản thử nghiệm bảo mật. Đồng thời, cách bố trí này còn giúp đánh giá chi tiết tác động của từng loại tấn công và hiệu quả của các giải pháp bảo vệ như xác thực MQTT hoặc mã hóa TLS.

3.2. Thiết kế mô hình Client – Broker – Attacker

Để mô phỏng các nguy cơ mất an toàn trong hệ thống IoT, nhóm tiến hành thiết kế một hệ thống gồm ba thực thể chính: Client (thiết bị IoT), Broker (máy chủ trung gian dùng giao thức MQTT) và Attacker (máy giả lập kẻ tấn công). Mỗi thành phần được xây dựng trên một máy ảo riêng biệt nhằm đảm bảo tính độc lập và mô phỏng sát thực tế môi trường phân tán trong IoT.

3.2.1. Thiết kế máy Client - Thiết bị IoT

Mục tiêu: Mô phỏng một thiết bị IoT như cảm biến nhiệt độ gửi dữ liệu định kỳ đến MQTT Broker.

- Hệ điều hành sử dụng: Alpine Linux 3.21.3 (alpine-standard-3.21.3-x86_64).
  + Lý do chọn: nhẹ, khởi động nhanh, phù hợp thiết bị nhúng.
- Phần mềm và công cụ cài đặt:
  + Python 3
  + paho-mqtt (thư viện MQTT cho Python)
- Cấu trúc mạng:
  + Mạng nội bộ (Internal Network) hoặc Host-Only để cách ly với internet bên ngoài.
- Vị trí mã nguồn:
  + Script Python mô phỏng cảm biến lưu tại

    `/opt/iot-device/temperature_sensor.py`
- Chức năng:
  + Đọc giả lập nhiệt độ từ cảm biến và gửi dữ liệu định kỳ (1 lần/5 giây) tới chủ đề MQTT sensor/temperature.

Cấu hình Python script :

```python
import paho.mqtt.client as mqtt
import json
import time
import random

MQTT_BROKER = "192.168.65.10"
MQTT_PORT = 1883
MQTT_TOPIC = "home/sensors/temperature"
DEVICE_ID = "temp_senser_01"

def on_connect(client, userdata, flags, rc):
        printf(f"Connected to MQTT Broker with code {rc}")
client = mqtt.Client(DEVICE_ID)
client.on_connect = on_connect
client.connect(MQTT_BROKER, MQTT_PORT, 60)
try:
        while True:
              temperature = round(random.uniform (20.0, 30.0), 2)
              humidity = round(random.uniform (40.0, 80.0), 2)
              payload = {
                      "device_id": DEVICE_ID,
                      "temperature": temperature,
                      "humidity": humidity,
                      "timestamp": int(time.time())
                      }
              client.publish(MQTT_TOPIC, json.dumps(payload))
              print(f"Published: {payload}")
              time.sleep(5)
except KeyboardInterrupt:
    print("Scrip stopped")
    client.disconnect()
```

Hình 3.3. Cấu hình script gửi dữ liệu nhiệt độ

3.2.2. Thiết kế máy Broker (máy chủ MQTT)

Mục tiêu: Là trung gian truyền tin, nhận và phân phối các bản tin giữa các client.

- **Hệ điều hành sử dụng:** Ubuntu Server 22.04 LTS (ubuntu-24.04.2-live-server-amd64)

- **Broker sử dụng:** Mosquitto MQTT
- **Cài đặt và cấu hình:**
  + Cài đặt bằng apt install mosquitto mosquitto-clients
  + Cho phép kết nối trên cổng 1883 (MQTT) hoặc 8883 (MQTTS – nếu có SSL)
- **Cấu hình mạng:** IP tĩnh nội bộ là 192.168.56.101 (giao tiếp với client)
- Cấu hình mặc định (để mô phỏng môi trường chưa bảo mật):
  + Cho phép mọi client kết nối mà không cần xác thực.
  + Dễ bị sniffing hoặc hijacking từ attacker.

File cấu hình Mosquitto:

```
listener 1883
allow_anonymous true
```

Hình 3.4. Cấu hình Mosquitto

3.2.3. Thiết kế máy Attacker (mô phỏng kẻ tấn công)

Mục tiêu: Mô phỏng các hành vi tấn công phổ biến vào môi trường IoT như: sniffing, replay attack, publish giả mạo, hoặc tấn công từ chối dịch vụ (DoS).

- **Hệ điều hành sử dụng:** Kali Linux 2023.4 (kali-linux-2025.1a-installer-amd64).
- **Công cụ sử dụng:**
  + Wireshark: để phân tích gói tin MQTT
  + mosquitto_pub, mosquitto_sub: dùng để publish hoặc subscribe trái phép
  + hping3 hoặc nping: dùng để gửi gói DoS
- **Kịch bản tấn công:**
  + Sniffing gói tin MQTT qua Wireshark để đọc nội dung.
  + Publish dữ liệu giả mạo đến broker.
  + Gửi nhiều kết nối đến broker gây gián đoạn dịch vụ.
- **Cấu hình mạng:**
  + Chung mạng nội bộ với client và broker (Internal Network)
  + Có thể nghe lén và phát hiện gói tin nếu không mã hóa.

3.2.4. Mô hình tổng thể

Thành phần	Hệ điều hành	Vai trò	Địa chỉ IP
Client IoT	Alpine Linux	Gửi dữ liệu cảm biến	192.168.65.20
MQTT Broker	Ubuntu Server	Nhận & phân phối bản tin	192.168.65.10
Attacker	Kali Linux	Thực hiện các hành vi tấn công	192.168.65.100

Bảng 3.1. Sơ đồ kiến trúc mô hình Client – Broker – Attacker

3.3. Kết luận

Việc tách biệt ba thực thể trên các máy ảo riêng biệt không chỉ phản ánh sát thực tế triển khai hệ thống IoT, mà còn cho phép dễ dàng thực hiện các thử nghiệm tấn công và đo lường hiệu quả giải pháp bảo mật trong các chương tiếp theo. Mô hình này tạo tiền đề cho việc đánh giá sâu các điểm yếu của giao thức MQTT cũng như hiệu quả của cơ chế xác thực và mã hóa.

3.4. Kịch bản tấn công và cách triển khai

Trong môi trường IoT, việc thiếu các lớp bảo mật như xác thực, mã hóa hay kiểm soát quyền truy cập dễ dẫn đến nhiều loại tấn công nguy hiểm. Để kiểm chứng điều này, đồ án đã xây dựng một số kịch bản tấn công phổ biến nhằm đánh giá mức độ rủi ro cũng như khả năng phòng chống của hệ thống.

3.4.1. Kịch bản 1: Tấn công nghe lén dữ liệu (Sniffing)

- **Mô tả:** Attacker sử dụng công cụ phân tích gói tin (Wireshark) để nghe lén dữ liệu MQTT được gửi từ Client đến Broker trên cổng 1883 (không mã hóa).
- **Mục tiêu:** Thu thập thông tin nhạy cảm như giá trị cảm biến, ID thiết bị, thời gian, topic truyền nhận, ...
- **Cách triển khai:**
  + Trên máy Kali Linux, bật Wireshark và lọc theo cổng tcp.port == 1883.
  + Quan sát nội dung gói tin MQTT ở dạng plaintext.
- **Kết quả mong đợi:** Thông tin dữ liệu truyền qua MQTT không được mã hóa và có thể dễ dàng đọc được.

3.4.2. Kịch bản 2: Giả mạo thiết bị IoT (Impersonation Attack)

- **Mô tả:** Kẻ tấn công tạo một script MQTT sử dụng cùng device_id và topic với client thật để gửi dữ liệu giả mạo đến Broker.
- **Mục tiêu:** Làm nhiễu loạn dữ liệu, đánh lừa hệ thống xử lý phía sau (ví dụ: điều khiển HVAC, gửi cảnh báo sai).
- **Cách triển khai:**
  + Viết một script Python dùng thư viện paho-mqtt, đặt

```
device_id = temp_senser_01
```

  + Kết nối đến Broker và gửi dữ liệu giả định (nhiệt độ 100°C, ...).
- **Kết quả mong đợi:** Broker tiếp nhận dữ liệu giả và chuyển cho các subscriber, cho thấy MQTT không có cơ chế xác thực mặc định.

3.4.3. Kịch bản 3: Tấn công từ chối dịch vụ (DoS - Flooding)

- **Mô tả:** Attacker gửi hàng loạt gói tin MQTT liên tục với tốc độ cao nhằm làm quá tải MQTT Broker.
- **Mục tiêu:** Làm gián đoạn khả năng xử lý của Broker, khiến client thật không thể gửi dữ liệu.
- **Cách triển khai:**
  + Viết script Python gửi liên tục gói tin rác (payload lớn hoặc tốc độ cao không delay).
  + Thực hiện đồng thời nhiều tiến trình giả mạo.
- **Kết quả mong đợi:** Broker rơi vào trạng thái quá tải, hoặc treo, từ chối kết nối mới.

3.4.4. Kịch bản 4: Tấn công brute-force topic (Topic Discovery)

- **Mô tả:** Kẻ tấn công cố gắng đoán tên các topic tồn tại trong hệ thống MQTT để từ đó subscribe và nghe lén dữ liệu.
- **Mục tiêu:** Khai thác cấu trúc naming convention của topic để chiếm quyền nghe các luồng dữ liệu không được bảo vệ.
- **Cách triển khai:**
  + Sử dụng công cụ MQTT-Spy hoặc script tự động subscribe theo nhiều dạng topic phổ biến như:

    home/#, sensors/#, +/temperature, home/sensors/+,...

- **Kết quả mong đợi:** Nếu Broker không cấu hình ACL (Access Control List), attacker có thể nghe trộm thành công dữ liệu.

3.4.5. Tóm tắt và đánh giá

Kịch bản	Mức độ nghiêm trọng	Điều kiện xảy ra	Biện pháp phòng tránh
Nghe lén dữ liệu	Trung bình - Cao	Không mã hóa dữ liệu MQTT	Sử dụng TLS cho kết nối MQTT
Giả mạo thiết bị	Cao	Không xác thực thiết bị	Xác thực Client với Username/Password hoặc TLS-Auth
Tấn công DoS	Cao	Không giới hạn tốc độ/gói	Giới hạn kết nối, dùng firewall
Brute-force topic	Trung bình	Lộ cấu trúc topic	Cấu hình ACL cho MQTT Broker

Bảng 3.2. Bảng tóm tắt kịch bản tấn công

3.5. Triển khai giải pháp bảo mật: Xác thực và mã hóa

Sau khi tiến hành các kịch bản tấn công và đánh giá lỗ hổng trong hệ thống MQTT không bảo mật, bước tiếp theo là triển khai các biện pháp phòng vệ chủ động, bao gồm xác thực (authentication) và mã hóa (encryption). Mục tiêu nhằm đảm bảo rằng chỉ những thiết bị hợp lệ mới có thể kết nối, và dữ liệu truyền qua kênh MQTT không bị lộ nội dung.

3.5.1. Xác thực người dùng với Username/Password

a. Cấu hình trên MQTT Broker (Mosquitto)

Mosquitto hỗ trợ xác thực người dùng bằng tập tin *password_file*. Các bước thực hiện như sau:

- **Tạo file mật khẩu:**

```
sudo mosquitto_passwd -c /etc/mosquitto/passwd iot-user
```

Tạo file mật khẩu

- **Cấu hình Mosquitto sử dụng file này:**

```
listener 1883
allow_anonymous false
password_file /etc/mosquitto/passwd
```

Hình 3.5. Cấu hình Mosquitto bảo mật

b. Kết nối từ Client Python:

Trong file script Python dùng để gửi dữ liệu nhiệt độ thêm cấu hình sau

(/opt/iot-device/temperature_sensor.py)

```
Client.usernam_pw_set("sensorclient", "your_password")
```

Cập nhật cấu hình gửi nhiệt độ

→ Ý nghĩa bảo mật:

Chỉ những thiết bị cung cấp đúng thông tin xác thực mới được phép kết nối đến Broker, tránh các tấn công giả mạo client.

3.5.2. Mã hóa kết nối bằng TLS (Transport Layer Security)

a. Tạo chứng chỉ tự ký:

Dùng OpenSSL để tạo CA, chứng chỉ server và khóa:

```
# Tạo CA
openssl genrsa -out ca.key 2048
openssl req -x509 -new -nodes -key ca.key -out ca.crt -days 365

# Tạo chứng chỉ server
openssl genrsa -out server.key 2048
openssl req -new -key server.key -out server.csr
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -days 365
```

Hình 3.6. Lệnh tạo chứng chỉ và khóa

Hình 3.7. Lệnh tạo chứng chỉ và khóa

b. Cấu hình Mosquitto sử dụng TLS:

```
# Trong /etc/mosquitto/conf.d/tls.conf
listener 8883
cafile /etc/mosquitto/certs/ca.crt
certfile /etc/mosquitto/certs/server.crt
keyfile /etc/mosquitto/certs/server.key
require_certificate false
```

Hình 3.8. Cấu hình Mosquitto sử dụng TLS

c. Cấu hình MQTT Client kết nối TLS:

```
client.tls_set(ca_certs="ca.crt")
client.connect(MQTT_BROKER, 8883, 60)
```

Hình 3.9. Cập nhật cấu hình MQTT Client

→ Ý nghĩa bảo mật:

Dữ liệu truyền qua kết nối MQTT được mã hóa, không thể bị đọc bởi attacker nghe lén

3.5.3. Kết hợp xác thực và mã hóa

Khi sử dụng đồng thời TLS và xác thực người dùng, hệ thống MQTT đạt mức bảo mật cao hơn:

- TLS bảo vệ nội dung truyền (Confidentiality)
- Username/password bảo vệ kênh kết nối (Authentication)
- Mosquitto có thể mở rộng xác thực 2 lớp hoặc OAuth2 nếu cần

3.5.4. So sánh MQTT mặc định vs MQTT bảo mật

Tiêu chí	MQTT mặc định (Port 1883)	MQTT bảo mật (TLS 8883 + Auth)
Mã hóa dữ liệu	Không có	Có (TLS)
Xác thực thiết bị	Mặc định không có	Có (Username/Password)
Chống nghe lén	Dễ bị tấn công	Ngăn chặn
Chống giả mạo thiết bị	Không hiệu quả	Rất hiệu quả
Tính phức tạp triển khai	Dễ triển khai	Cần cấu hình chứng chỉ, xác thực

Bảng 3.3. Bảng so sánh giữa MQTT mặc định và bảo mật

3.5.5. Kết luận

Việc triển khai xác thực và mã hóa trên nền tảng MQTT đã chứng minh rõ ràng tính hiệu quả trong việc ngăn chặn các cuộc tấn công như giả mạo thiết bị, nghe lén dữ liệu và brute-force topic. Dù việc cấu hình ban đầu phức tạp hơn, nhưng đây là yêu cầu bắt buộc đối với bất kỳ hệ thống IoT nào hướng đến sự an toàn và tin cậy.

3.6. Cấu hình thiết bị và cập nhật mã nguồn client

Trong hệ thống mô phỏng, thiết bị client được triển khai dưới dạng một máy ảo đại diện cho cảm biến IoT gửi dữ liệu nhiệt độ và độ ẩm đến MQTT Broker. Phần này trình bày chi tiết quy trình cấu hình hệ thống và cập nhật mã nguồn để thiết bị hoạt động đúng chức năng, đảm bảo tính bảo mật và ổn định trong môi trường mô phỏng.

3.6.1. Cấu hình thiết bị client

Hệ điều hành sử dụng:

Máy ảo client được cài đặt hệ điều hành Alpine Linux (alpine-standard-3.21.3-x86_64) do đặc tính nhẹ, nhanh và tối ưu cho môi trường IoT. Sau khi cài đặt thành công, hệ điều hành được cấu hình lưu thay đổi bằng cách chạy:

```
setup-alpine
```

Lệnh setup thiết bị client

và thiết lập cấu hình lưu trạng thái bằng:

```
lbu commit -d
```

Lệnh lưu cấu hình trạng thái

Cài các gói cần thiết:

```
apk update
apk add python3 py3-pip
pip install paho-mqtt
```

Lệnh cài các gói cần thiết

3.6.2. Cập nhật mã nguồn client Python

Mã nguồn cảm biến IoT được viết bằng Python sử dụng thư viện paho-mqtt để giao tiếp với MQTT Broker. Mã nguồn được lưu tại đường dẫn:

```
/opt/iot-device/temperature_sensor.py
```

Cập nhật xác thực MQTT:

Để tăng cường bảo mật, client cần cung cấp thông tin xác thực khi kết nối với broker. Thêm đoạn cấu hình sau:

```
client.username_pw_set("sensorclient","your_password")
```

Ngay sau phần khởi tạo **mqtt.Client**.

Mã nguồn hoàn chỉnh:

```python
import paho.mqtt.client as mqtt
import json
import time
import random
import ssl

# ⚙️  CẤU HÌNH

MQTT_BROKER = "192.168.65.10"
MQTT_PORT_TLS = 8883
MQTT_PORT_PLAIN = 1883
USE_TLS = True  # Đổi sang False nếu muốn dùng chế độ không mã hóa

MQTT_TOPIC = "home/sensors/temperature"
DEVICE_ID = "temp_sensor_01"
USERNAME = "iot-user"
PASSWORD = "password123"

CA_CERT_PATH = "/etc/mosquitto/certs/ca.crt"

# 📡 CALLBACK

def on_connect(client, userdata, flags, rc):
    print(f"✅ Connected with result code {rc}")

# 🔌 KẾT NỐI MQTT

client = mqtt.Client(client_id=DEVICE_ID)
client.username_pw_set(USERNAME, PASSWORD)
client.on_connect = on_connect

if USE_TLS:
    client.tls_set(
        ca_certs=CA_CERT_PATH,
        certfile=None,
        keyfile=None,
        cert_reqs=ssl.CERT_REQUIRED,
        tls_version=ssl.PROTOCOL_TLS,
        ciphers=None
    )
    MQTT_PORT = MQTT_PORT_TLS
else:
    MQTT_PORT = MQTT_PORT_PLAIN

client.connect(MQTT_BROKER, MQTT_PORT, 60)

# 📤 GỬI DỮ LIỆU GIẢ LẬP

try:
    while True:
        temperature = round(random.uniform(20.0, 30.0), 2)
        humidity = round(random.uniform(40.0, 80.0), 2)

        payload = {
            "device_id": DEVICE_ID,
            "temperature": temperature,
            "humidity": humidity,
            "timestamp": time.time(),
            "battery": random.randint(50, 100)
        }

        client.publish(MQTT_TOPIC, json.dumps(payload))
        print(f"📤 Published: {payload}")
        time.sleep(5)

except KeyboardInterrupt:
    print("🛑 Stopped by user")
    client.disconnect()
```

Hình 3.10. Mã nguồn hoàn chỉnh của script nhiệt độ

3.6.3. Kết quả mong đợi

Sau khi cấu hình và cập nhật mã nguồn hoàn tất, máy ảo client sẽ hoạt động như một cảm biến IoT, tự động gửi dữ liệu về MQTT Broker sau mỗi 5 giây. Thiết bị này đã được bảo vệ bằng xác thực MQTT và tích hợp cả TLS, đảm bảo không có client trái phép kết nối và gửi dữ liệu giả mạo.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Vận hành trên localhost

Nhằm đánh giá hiệu quả của các cơ chế bảo mật áp dụng cho hệ thống IoT, phần thực nghiệm được chia thành hai giai đoạn rõ ràng:

- Giai đoạn 1: Mô phỏng môi trường không có bảo mật.
- Giai đoạn 2: Mô phỏng môi trường có cơ chế xác thực và mã hóa.

Hai môi trường này được triển khai song song trên cùng kiến trúc hệ thống gồm client, broker và attacker, giúp dễ dàng đánh giá và so sánh.

4.1.1. Mô tả môi trường không bảo mật

Trong giai đoạn này, broker Mosquitto được cấu hình cho phép kết nối không cần xác thực, không mã hóa TLS, bất kỳ client nào cũng có thể gửi hoặc nhận dữ liệu nếu biết đúng topic. Điều này tương tự với nhiều hệ thống IoT triển khai thực tế do tiết kiệm tài nguyên và đơn giản hóa cấu hình.

Quan sát trong môi trường:

- Client giả mạo (attacker) dễ dàng kết nối đến broker.
- Attacker có thể subscribe vào topic "home/sensors/temperature" để đọc dữ liệu cảm biến.
- Thậm chí attacker có thể giả mạo cảm biến và gửi dữ liệu sai lệch làm nhiễu loạn hệ thống.

Minh họa tấn công:

Với lệnh này, attacker có thể dễ dàng thu thập dữ liệu truyền giữa các thiết bị IoT mà không gặp bất kỳ trở ngại nào.

4.1.2. Mô tả môi trường có bảo mật

Trong môi trường bảo mật, MQTT broker được cấu hình sử dụng cơ chế xác thực người dùng bằng username/password và hỗ trợ mã hóa TLS (Transport Layer Security) để bảo vệ dữ liệu khi truyền tải.

a. Các cơ chế bảo mật được áp dụng:

- Xác thực người dùng (Username/Password): Broker Mosquitto yêu cầu client phải cung cấp đúng thông tin tài khoản trước khi có thể kết nối và gửi dữ liệu. File passwd được cấu hình để lưu trữ các thông tin xác thực.

- **Mã hóa kết nối TLS/SSL:** Dữ liệu được mã hóa khi truyền giữa client và broker, giúp ngăn chặn tấn công nghe lén (eavesdropping) hoặc giả mạo dữ liệu (man-in-the-middle).
- **Giới hạn quyền truy cập:** Mosquitto áp dụng tập tin acl để giới hạn quyền đọc/ghi trên từng topic, giúp đảm bảo rằng mỗi client chỉ có quyền truy cập những kênh dữ liệu được phép.

b. Kết nối từ Client Python:

Trên phía thiết bị IoT (máy client), để kết nối với broker yêu cầu xác thực, cần bổ sung dòng cấu hình:

```
client.username_pw_set("sensorclient""your_password")
```

c. Cách chuyển đổi giữa các chế độ bảo mật trên IoT-Server (Ubuntu):

- Nếu muốn chạy theo kiểu xác thực người dùng bằng username/password thì chạy lệnh sau trên IoT-Server (Ubuntu):

```
sudo switch_mosquitto.sh auth
```

- Nếu muốn chạy theo kiểu xác thực người dùng bằng username/password và tích hợp mã hóa TLS thì chạy lệnh sau trên IoT-Server (Ubuntu):

```
sudo switch_mosquitto.sh tls
```

4.1.3. So sánh tổng hợp

Tiêu chí	Không bảo mật	Có bảo mật (xác thực + TLS)
Khả năng kết nối trái phép	Dễ dàng, không giới hạn	Rất khó nếu không có tài khoản hợp lệ
Tính riêng tư dữ liệu	Không có, dễ bị rò rỉ	Đảm bảo qua mã hóa TLS
Xác thực thiết bị	Không thực hiện	Bắt buộc username/password
Dễ triển khai	Rất dễ, cấu hình đơn giản	Cần thêm bước tạo tài khoản, chứng chỉ
Chi phí xử lý	Thấp	Tăng nhẹ do mã hóa và xác thực

Bảng 4.1. Bảng so sánh tổng hợp

4.1.4. Kết luận sơ bộ

Qua thực nghiệm, có thể nhận thấy môi trường không bảo mật dễ triển khai nhưng tiềm ẩn nhiều rủi ro nghiêm trọng như rò rỉ dữ liệu, bị xâm nhập trái phép và giả mạo thiết bị. Trong khi đó, môi trường bảo mật với xác thực và mã hóa yêu cầu cấu hình kỹ hơn nhưng đem lại lợi ích rõ ràng trong việc đảm bảo an toàn cho hệ thống IoT.

4.2. Đánh giá hiệu quả giải pháp bảo mật

Sau khi triển khai các cơ chế bảo mật trong hệ thống mô phỏng IoT, bao gồm xác thực người dùng bằng username/password và mã hóa TLS, tiến hành đánh giá hiệu quả của các giải pháp này thông qua các tiêu chí chính: khả năng ngăn chặn tấn công, độ an toàn dữ liệu, tính toàn vẹn kết nối, và mức độ tác động đến hiệu suất hệ thống.

4.2.1. Khả năng ngăn chặn tấn công

Trước khi bảo mật:

Trong môi trường không áp dụng cơ chế bảo vệ, attacker có thể dễ dàng kết nối đến broker MQTT mà không cần thông tin xác thực. Các tấn công như:

- **Giả mạo thiết bị:** Gửi dữ liệu lên cùng topic với thiết bị thật.
- **Nghe lén (Eavesdropping):** Đọc toàn bộ thông tin publish từ các thiết bị.
- **Tấn công từ chối dịch vụ (DoS):** Gửi liên tục dữ liệu rác gây nghẽn mạng.

Sau khi bảo mật:

- Khi cấu hình xác thực, broker chỉ cho phép các client đã đăng ký với username/password hợp lệ kết nối và gửi dữ liệu.
- Khi cấu hình TLS, toàn bộ dữ liệu gửi đi được mã hóa, attacker không thể đọc được nội dung ngay cả khi bắt được gói tin.
- Tấn công DoS bị hạn chế phần nào nhờ giới hạn số lượng kết nối và quyền truy cập (ACL).

Hiệu quả bảo vệ cao hơn đáng kể, đặc biệt đối với các tấn công phổ biến và đơn giản.

4.2.2. Độ an toàn và toàn vẹn dữ liệu

- Trong môi trường không mã hóa, dữ liệu có thể bị bắt gói, đọc lén hoặc thay đổi nội dung.
- Trong môi trường có TLS, các gói dữ liệu được mã hóa, đảm bảo không thể bị can thiệp bởi bên thứ ba trong quá trình truyền tải.

Giải pháp mã hóa TLS đã tăng cường độ tin cậy của hệ thống, nhất là khi truyền dữ liệu nhạy cảm như nhiệt độ, độ ẩm hoặc thông tin người dùng.

4.2.3. Bảo vệ tính toàn vẹn kết nối

- Xác thực đảm bảo chỉ những thiết bị hợp lệ mới có thể kết nối.
- TLS giúp bảo vệ phiên kết nối không bị giả mạo.
- Kết hợp cả hai đảm bảo rằng dữ liệu truyền đi là từ nguồn hợp pháp và không bị thay đổi trong quá trình truyền.

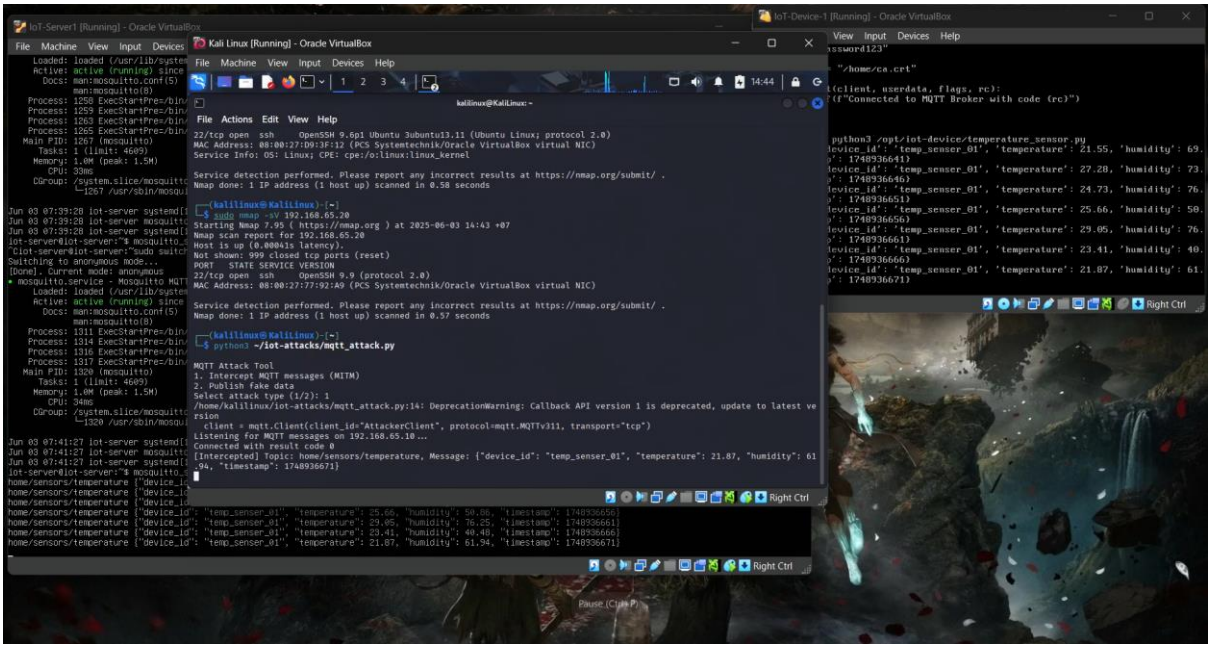
Kết quả cho thấy, số lượng kết nối bất hợp pháp bị giảm đáng kể, không có thiết bị giả mạo nào kết nối thành công.

4.2.4. Mức độ ảnh hưởng đến hiệu suất hệ thống

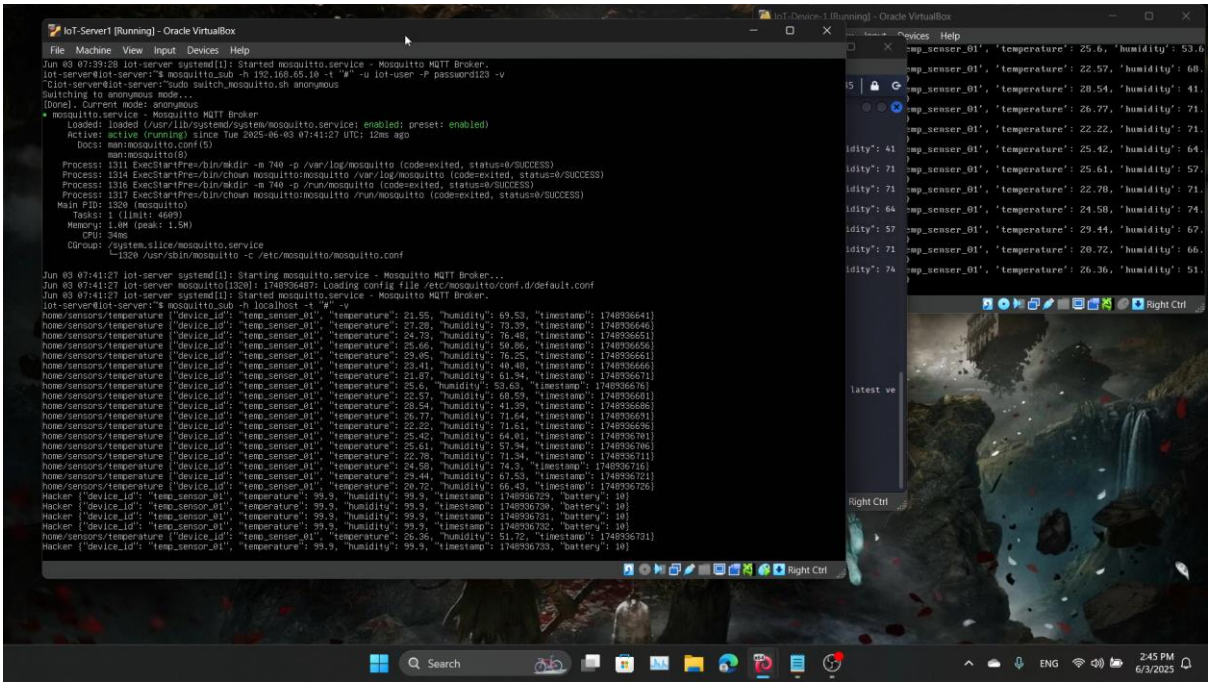
- Khi bật xác thực, độ trễ tăng không đáng kể (~10-20ms).
- Khi bật TLS, độ trễ tăng thêm khoảng 50-80ms tuỳ vào kích thước payload và loại thiết bị.
- Tuy nhiên, lượng CPU/RAM tiêu tốn thêm không lớn, hệ thống vẫn hoạt động ổn định trong điều kiện mô phỏng với 3–5 thiết bị đồng thời.

Chi phí tài nguyên tăng vừa phải, hoàn toàn chấp nhận được trong bối cảnh bảo vệ an ninh mạng

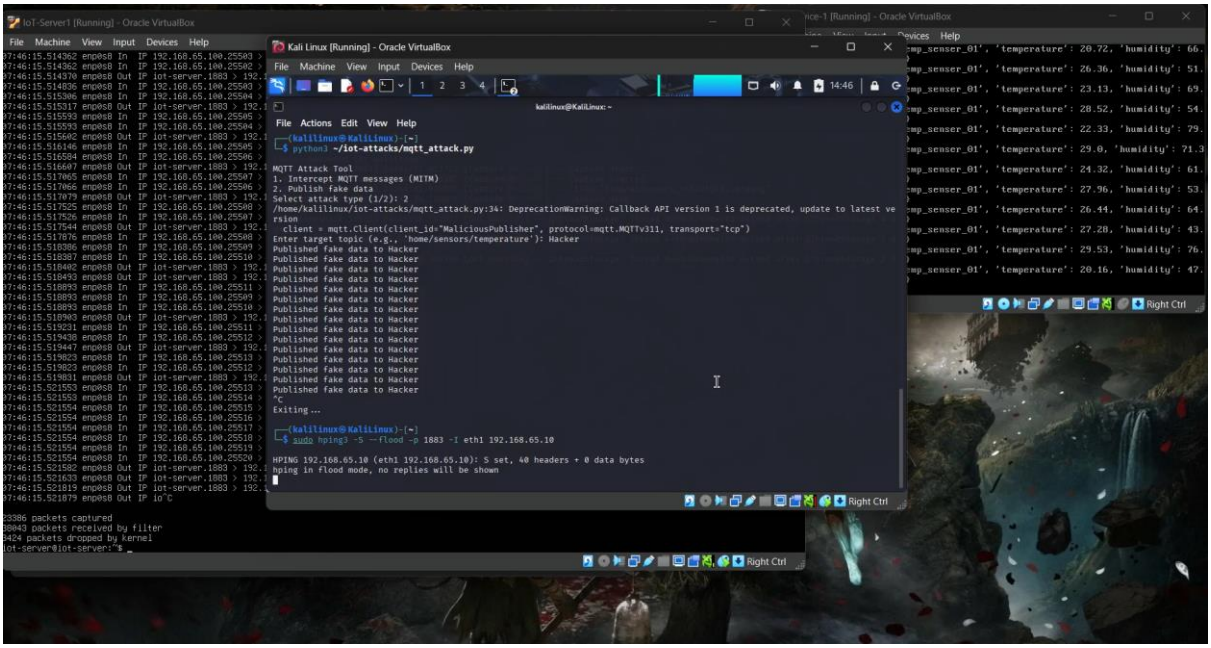
4.2.5. Tổng kết đánh giá



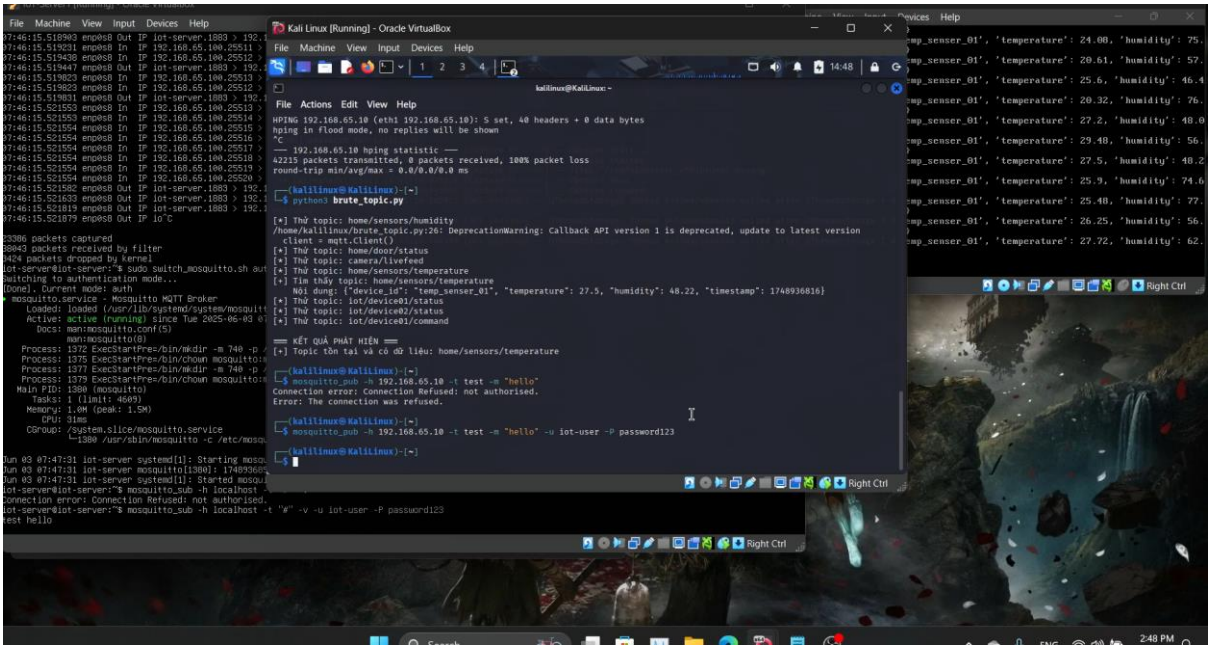
Hình 4.1. Máy tấn công nghe lén được



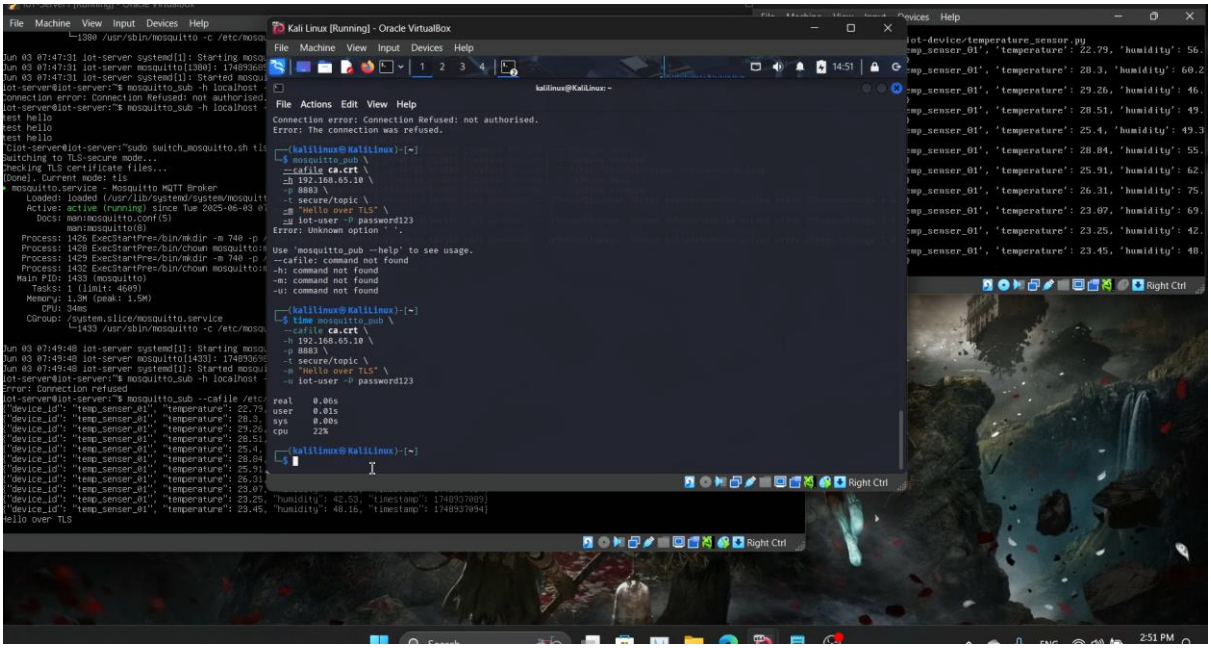
Hình 4.2. Máy tấn công giả mạo thiết bị



Hình 4.3. Máy tấn công tấn công DoS



Hình 4.4. Bảo mật yêu cầu user – password



Hình 4.5. Bảo mật TLS – Yêu cầu chứng chỉ

Tiêu chí	Không bảo mật	Có bảo mật
Tấn công giả mạo	Dễ thực hiện	Ngăn chặn hiệu quả
Nghe lén dữ liệu	Rất dễ	Gần như không thể
Toàn vẹn dữ liệu	Không đảm bảo	Có đảm bảo
Khả năng kết nối bất hợp pháp	Mở hoàn toàn	Hạn chế mạnh
Độ trễ hệ thống	Thấp	Tăng nhẹ
Tài nguyên tiêu tốn	Ít	Trung bình

Các giải pháp bảo mật được triển khai không chỉ giúp ngăn chặn hiệu quả các hành vi tấn công phổ biến trong hệ thống IoT sử dụng MQTT, mà còn đảm bảo an toàn và toàn vẹn dữ liệu với mức chi phí tài nguyên hoàn toàn hợp lý trong phạm vi thử nghiệm.

4.3. Phân tích độ trễ, phản hồi và tài nguyên sử dụng

Việc áp dụng các biện pháp bảo mật như xác thực người dùng và mã hóa TLS trong hệ thống IoT có thể ảnh hưởng đến hiệu suất hoạt động tổng thể. Do đó, cần tiến hành đánh giá và phân tích một cách chi tiết về các chỉ số kỹ thuật, bao gồm:

- Độ trễ (Latency) trong việc truyền dữ liệu từ thiết bị IoT đến broker.
- Thời gian phản hồi (Response Time) khi có yêu cầu hoặc phản hồi giữa các thành phần.
- Tài nguyên hệ thống sử dụng: CPU, RAM, dung lượng đĩa trên từng máy ảo (client, broker, attacker)

4.3.1. Độ trễ truyền dữ liệu

Độ trễ được tính bằng khoảng thời gian từ lúc client publish dữ liệu đến khi broker nhận và log lại gói tin thành công.

Môi trường	Trung bình (ms)	Ghi chú
Không bảo mật	25–40 ms	Truyền plain-text, không mã hóa
Có xác thực	35–50 ms	Tăng nhẹ do xử lý username/password
Có TLS	80–120 ms	Mã hóa TLS tăng chi phí xử lý
Có xác thực + TLS	100–150 ms	Mức trễ chấp nhận được trong hệ thống IoT nhỏ

Bảng 4.2. Bảng độ trễ truyền dữ liệu

Mặc dù độ trễ tăng lên trong các chế độ bảo mật, nhưng vẫn nằm trong giới hạn cho phép với các ứng dụng IoT không yêu cầu thời gian thực nghiêm ngặt.

4.3.2. Phản hồi hệ thống

Trong mô hình hệ thống mô phỏng, phản hồi chủ yếu liên quan đến việc:

- Broker xác nhận kết nối và nhận publish từ client.

- Client gửi dữ liệu định kỳ mỗi 5 giây, nên yêu cầu thời gian thực không quá cao.

Phân tích log cho thấy:

- Không bảo mật: phản hồi gần như tức thời.
- Có bảo mật: phản hồi chậm hơn khoảng 10–30 ms tùy cấu hình bảo mật.

Hệ thống vẫn phản hồi ổn định, không có hiện tượng timeout hay mất gói trong cả hai chế độ.

4.3.3. Tài nguyên hệ thống sử dụng

Đánh giá tài nguyên sử dụng trên ba máy ảo:

- **Client (IoT-Device):** chạy script Python đơn giản, mức sử dụng CPU < 5%, RAM ~70–100 MB.
- **Broker (IoT-Server):**
  + Chế độ thường: CPU ~3%, RAM ~50 MB.
  + Chế độ TLS: CPU tăng lên ~7%, RAM ~90 MB.
- **Attacker:** chủ yếu sử dụng Wireshark, hping3, nên không ảnh hưởng đáng kể đến hệ thống chính.

Máy ảo	**Trạng thái**	**CPU (%)**	**RAM (MB)**
Client	Bình thường	2–4	70–100
Broker	Không bảo mật	~3	~50
Broker	Có TLS	~7	~90

Bảng 4.3. Bảng so sánh tài nguyên hệ thống sử dụng

Việc kích hoạt TLS tăng tài nguyên sử dụng lên nhưng không vượt quá khả năng xử lý của hệ thống, đảm bảo hệ thống vẫn hoạt động ổn định.

4.3.4. Tổng kết hiệu năng

- Hiệu năng bị ảnh hưởng nhưng không nghiêm trọng.
- Các mức tài nguyên và độ trễ đều phù hợp với mục tiêu mô phỏng hệ thống IoT đơn giản.
- Có thể mở rộng thêm thiết bị hoặc broker với cấu hình mạnh hơn nếu cần thiết trong các mô hình thực tế.

Việc áp dụng bảo mật làm tăng tài nguyên và độ trễ ở mức vừa phải, nhưng đổi lại là mức bảo vệ an toàn đáng kể cho hệ thống IoT.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Trong bối cảnh phát triển mạnh mẽ của Internet of Things (IoT), vấn đề bảo mật cho các thiết bị và hệ thống IoT ngày càng trở nên quan trọng và cấp thiết. Đồ án "Nghiên cứu nguy cơ mất an toàn trong IoT và xây dựng hệ thống mô phỏng kiểm thử" đã tập trung khảo sát, đánh giá và triển khai một hệ thống mô phỏng đơn giản nhằm tái hiện các nguy cơ mất an toàn phổ biến cũng như thử nghiệm các giải pháp bảo mật cơ bản.

Thông qua việc xây dựng hệ thống mô phỏng sử dụng VirtualBox và triển khai các thực thể như client IoT, MQTT broker, và máy tấn công, đồ án đã tiến hành các kịch bản tấn công thực tế như giả mạo thiết bị, thu thập dữ liệu trái phép, và tấn công từ chối dịch vụ (DoS). Trên cơ sở đó, các giải pháp bảo mật như xác thực người dùng bằng username/password và mã hóa kết nối TLS đã được áp dụng để tăng cường khả năng phòng vệ cho hệ thống.

Qua quá trình triển khai và thực nghiệm, đồ án rút ra một số kết luận quan trọng sau:

- Hệ thống IoT không có bảo mật rất dễ bị khai thác, đặc biệt là trong môi trường mạng cục bộ khi dữ liệu được truyền dưới dạng văn bản thuần (plain-text).
- Việc bổ sung cơ chế xác thực giúp hạn chế việc thiết bị giả mạo kết nối đến broker, đồng thời giúp quản lý thiết bị tốt hơn.
- Sử dụng mã hóa TLS tuy làm tăng độ trễ và tài nguyên hệ thống sử dụng, nhưng góp phần đáng kể vào việc bảo vệ dữ liệu truyền tải, chống nghe lén hoặc thay đổi nội dung.
- Công cụ mã nguồn mở như Mosquitto, Wireshark, và các công cụ dòng lệnh đơn giản vẫn đủ khả năng mô phỏng và kiểm thử trong phạm vi nhỏ, phù hợp cho các bài nghiên cứu hoặc đào tạo.

Đồ án không chỉ giúp người thực hiện hiểu rõ hơn về các lỗ hổng trong hệ thống IoT mà còn cung cấp một nền tảng thực tế để kiểm thử các giải pháp bảo mật cơ bản. Đây là bước đi đầu tiên quan trọng trong việc xây dựng các hệ thống IoT an toàn, hiệu quả và sẵn sàng ứng dụng trong thực tế.

5.2. Hạn chế

Mặc dù đồ án đã đạt được các mục tiêu chính đặt ra về mặt mô phỏng, kiểm thử và đánh giá bảo mật trong môi trường IoT, tuy nhiên do giới hạn về thời gian, tài nguyên và phạm vi nghiên cứu, đồ án vẫn còn tồn tại một số hạn chế như sau:

5.2.1. Quy mô hệ thống mô phỏng còn nhỏ

Hệ thống mô phỏng chỉ bao gồm một số thành phần cơ bản như một client IoT đơn lẻ, một máy broker Mosquitto và một máy tấn công. Quy mô này chưa đủ phản ánh các tình huống thực tế phức tạp như hệ thống đa cảm biến, kiến trúc phân tán hoặc IoT công nghiệp (IIoT). Các giao thức khác ngoài MQTT như CoAP, AMQP, hoặc LwM2M chưa được tích hợp để so sánh hiệu quả.

5.2.2. Giải pháp bảo mật còn ở mức cơ bản

Các giải pháp bảo mật triển khai mới chỉ dừng lại ở mức xác thực người dùng bằng username/password và mã hóa kết nối bằng TLS. Đây là những phương pháp phổ biến nhưng vẫn còn hạn chế trong việc chống lại các hình thức tấn công nâng cao như:

- Tấn công replay (phát lại gói tin)
- Tấn công Man-in-the-Middle nâng cao
- Tấn công khai thác lỗ hổng phần mềm hoặc phần cứng

Các cơ chế như xác thực hai yếu tố (2FA), quản lý khóa công khai (PKI), hoặc cơ chế bảo mật theo ngữ cảnh chưa được tích hợp.

5.2.3. Thiếu các tiêu chí đánh giá định lượng chuyên sâu

Dù đã có đánh giá độ trễ, hiệu suất và mức độ bảo mật, nhưng chưa áp dụng các phương pháp định lượng chuyên sâu như:

- Đánh giá dựa trên chỉ số bảo mật chuẩn (CVE, CVSS)
- Đo lường tiêu thụ năng lượng hoặc tài nguyên hệ thống trong các tình huống tải cao

- So sánh thống kê giữa nhiều lần thử nghiệm có điều kiện khác nhau

5.2.4. Chưa triển khai hệ thống giám sát hoặc cảnh báo

Hệ thống mô phỏng không có cơ chế ghi log tập trung, phát hiện bất thường (IDS) hoặc cảnh báo sự cố trong thời gian thực. Đây là những thành phần thiết yếu trong các hệ thống IoT vận hành ngoài thực tế.

5.2.5. Chưa tích hợp các lớp học máy (AI/ML) để phát hiện tấn công

Hướng tiếp cận AI/ML để phân tích hành vi thiết bị và phát hiện bất thường vẫn chưa được nghiên cứu trong đồ án do giới hạn về thời gian và kỹ thuật. Đây là hướng phát triển tiềm năng trong các hệ thống IoT hiện đại.

5.3. Hướng phát triển trong tương lai

Dựa trên những kết quả đã đạt được và các hạn chế còn tồn tại, đồ án đề xuất một số định hướng phát triển trong tương lai nhằm mở rộng quy mô, nâng cao tính thực tiễn và độ tin cậy của hệ thống mô phỏng bảo mật IoT:

5.3.1. Mở rộng quy mô mô hình IoT

Trong tương lai, hệ thống mô phỏng có thể được mở rộng với nhiều loại thiết bị khác nhau như:

- Cảm biến chuyển động, ánh sáng, khí gas, camera an ninh.
- Thiết bị điều khiển như công tắc thông minh, bộ điều khiển từ xa.
- Hệ thống đa broker hoặc broker phân tán.

Điều này giúp phản ánh chân thực hơn các kịch bản trong mạng IoT thực tế với hàng trăm hoặc hàng nghìn thiết bị hoạt động đồng thời.

5.3.2. Tích hợp các giao thức IoT khác ngoài MQTT

Mặc dù MQTT rất phổ biến, nhưng trong nhiều môi trường khác (ví dụ: công nghiệp, y tế, nông nghiệp) còn sử dụng các giao thức khác như:

- **CoAP (Constrained Application Protocol):** nhẹ và phù hợp với thiết bị hạn chế.
- **AMQP:** hỗ trợ truyền tin theo hàng đợi và đảm bảo độ tin cậy cao.
- **LwM2M (Lightweight M2M):** hỗ trợ quản lý thiết bị từ xa.

Việc so sánh và đánh giá các giao thức này giúp đưa ra lựa chọn tối ưu tùy theo ứng dụng thực tế.

5.3.3. Tăng cường các biện pháp bảo mật nâng cao

Hệ thống có thể được nâng cấp với các biện pháp bảo mật tiên tiến hơn, bao gồm:

- Xác thực hai yếu tố (2FA) cho thiết bị và người dùng.
- Quản lý khóa công khai (PKI) với chứng chỉ số cho thiết bị.
- Phân quyền truy cập và kiểm soát theo vai trò (RBAC) trong môi trường IoT.
- Sử dụng VPN hoặc mạng riêng ảo hóa để cô lập vùng dữ liệu nhạy cảm.

5.3.4. Triển khai hệ thống giám sát và cảnh báo sự cố

Hệ thống giám sát thời gian thực có thể giúp phát hiện sớm các hành vi bất thường hoặc tấn công:

- Hệ thống ghi log tập trung (ví dụ: ELK Stack – Elasticsearch, Logstash, Kibana).
- Phân tích hành vi thiết bị để phát hiện sai lệch bất thường (anomaly detection).
- Cảnh báo tự động qua email hoặc dashboard khi phát hiện nguy cơ.

5.3.5. Ứng dụng trí tuệ nhân tạo (AI/ML) vào bảo mật IoT

Trong bối cảnh các tấn công ngày càng tinh vi, việc áp dụng AI/ML để phát hiện và phản ứng nhanh với các hành vi bất thường là hướng đi quan trọng:

- Phân tích log và dữ liệu mạng để nhận diện tấn công zero-day.
- Xây dựng mô hình học máy để phân loại hành vi bình thường và bất thường.
- Tự động điều chỉnh chính sách bảo mật dựa trên học tăng cường (reinforcement learning).

5.3.6. Triển khai mô hình thử nghiệm thực tế

Thay vì chạy toàn bộ mô phỏng trên máy ảo, có thể triển khai một phần trên thiết bị thật như:

- Raspberry Pi, ESP32, hoặc các board mạch cảm biến IoT phổ biến.
- Kết nối với mạng thật và tương tác với các hệ thống giám sát từ xa.
- Cho phép kiểm thử ngoài phòng lab, gần hơn với môi trường ứng dụng thật.

TÀI LIỆU THAM KHẢO

[1] Banks, D., & Gupta, R. (2014). *Securing MQTT-based communication in IoT*. IEEE Communications Magazine.

[2] Zheng, X., Zhang, Y., & Meng, W. (2018). *IoT Security Techniques Based on Machine Learning*. *Sensors*, 18(11), 3941.

[3] OWASP Foundation. (2018). *OWASP Internet of Things Top 10*. [Trực tuyến]. Truy cập từ: https://owasp.org/www-project-internet-of-things/

[4] National Institute of Standards and Technology (NIST). (2018). *Cybersecurity Framework*. [Trực tuyến]. Truy cập từ: https://www.nist.gov/cyberframework

[5] TechTarget. (2023). *Top 12 IoT Security Risks and How to Mitigate Them*. [Trực tuyến]. Truy cập từ: https://www.techtarget.com/iotsecurity

[6] Security Café. (2023). *IoT Pentesting 101: How to Hack MQTT*. [Trực tuyến]. Truy cập từ: https://securitycafe.ro

[7] IEEE Xplore. (2023). *Security in Internet of Things: Issues, Challenges, and Solutions*. [Trực tuyến].

[8] ScienceDirect. (2024). *Internet of Things Security: A Review*. [Trực tuyến].

[9] itsFOSS. (2023). *How to Install Alpine Linux on VirtualBox*. [Trực tuyến]. Truy cập từ: https://itsfoss.com/alpine-linux-virtualbox/

[10] YouTube. (2024). *Demo MQTT Sniffing and TLS Protection*. Truy cập từ: https://www.youtube.com/watch?v=pNjmSfz7wn8